
Hierarchical Contrastive Pre-training of a Complex-Valued SegFormer for Polarimetric SAR Semantic Segmentation

End-of-studies project / Master Thesis — CentraleSupélec & SONDRA laboratory

Revised edition

Emmanuel BENICHO

Supervision: Jérémy Fix (CentraleSupélec)

Original report: 2025. Revised edition: September 2026, prepared by Emmanuel Benichou. The revised edition retains only the work carried out personally by Emmanuel Benichou within the project; see About this edition for the details.

Abstract

The code supporting the claims made in this report is available at <https://github.com/cziffras/sondra>.

Polarimetric synthetic aperture radar (PolSAR) is a promising technology that allows getting high resolution imagery of the Earth regardless of meteorological conditions, and luminosity. It delivers large amounts of complex-valued imagery, however pixel level annotations are scarce and costly. This work studies whether a network trained for semantic segmentation can learn useful representations from the unlabelled PolSAR acquisition data and be adapted to natively work with complex valued data. This second dimension of this work is of great importance as deep learning proved itself progressively more efficient when constructed as a mirror of the data it was meant to be working with, what is called inductive bias.

We work on the San Francisco ALOS-2 scene of the PolSF benchmark, whose labelled area covers only 7.7% of the acquisition. The model is a complex-valued re-implementation of SegFormer, a hierarchical vision transformer, built on the `torchcvnn` [8] library developed at CentraleSupélec, this report will put a heavy focus on explaining the consequences of such architectural choices. A hierarchical architecture was chosen due to the success of this type of architecture on semantic segmentation tasks reported in literature [21], [27] and [18] for a more fundamental approach.

Our method consists in developing a contrastive pretraining scheme for the encoder, which will exploit the 92% of data from the scene that carries no label, with SAR-specific augmentations designed not to break the identity between two views of a patch from the scene, working on SAR data raises several domain-specific questions, with less intuitive answers than for traditional RGB images: What can be a similarity measure between two complex-valued patches? What type of invariances should be used to design proper augmentations?

Because the encoder produces four feature maps at decreasing resolutions, we introduced one contrastive loss that is computed per stage, whence our central methodological question: how can we combine them? The intuition behind such loss comes from a simple fact: stages perform at different resolution levels and thus at different noise levels, not all should be penalized the same way for not learning consistent embeddings.

We compare two learnable weightings: a softmax weighting regularised towards the uniform distribution, and a formulation in which each stage owns a learnable temperature inside its InfoNCE likelihood, widely inspired by [15]. In the case of the second, we show that it is the probabilistically consistent way of weighting stages whose losses are in substance negative log-likelihoods. Both are evaluated by fine-tuning the pre-trained encoder and comparing it with a baseline initialized randomly and trained only on a growing portion of the labeled data. We aim to evaluate which initialization helps the model being more frugal in data consumption. Pre-training mainly reduces the variance of the supervised training and saves labels: with a quarter to a half of the labelled patches, the fine-tuned encoders match the baseline trained on all of them, while the two weightings, despite opposite behaviours, transfer equally well.

About this edition

This document is a revised edition of the end-of-studies project (PFE), standing as a Master Thesis in the CentraleSupélec curriculum report written in 2025 by **Emmanuel Benichou**, **Rodolphe Durand** and **Lazare Plisson-Arcos** at CentraleSupélec, in collaboration with the SONDRALABORATORY. CentraleSupélec’s cycle ingénieur curriculum does not formally propose students to write a personal master thesis, but an end-of-studies project instead which is conducted by a small group of 3 students. Both to tackle this personal credential issue regarding pre-requisites of most academic organizations worldwide and improve my own contribution to this work, I (Emmanuel Benichou) have authored the hereby revised version in the context of my application to a PhD position at NTNU.

Let me insist on the fact that this revised version retains only the parts of the project that Emmanuel Benichou carried out personally:

- the PolSF data pipeline, including the separation between the labelled area and the unlabelled pre-training regions;
- the complex-valued SAR augmentations used to build contrastive views;
- the complex-valued SegFormer;
- the multi-stage contrastive objectives and the study of their weighting;
- the PolSF segmentation experiments.

The study of MSTAR target classification with a complex Vision Transformer and the experiments on triplet losses and on a complex U-Net, which belonged to the original report, are thus not part of this edition.

The software was also improved for reproducibility and deeply edited for this edition. Several defects were found and corrected, the evaluation protocol was also substantially modified, and every experiment reported here was run anew. The code is available at <https://github.com/cziffra/sondra> (personal github account).

Contents

Abstract	i
About this edition	ii
1 Synthetic aperture radar and polarimetry	1
1.1 A coherent measurement	1
1.2 The "synthetic aperture" and radar imaging basics	2
1.2.1 Zoom on the notion of range resolution:	2
1.2.2 Zoom on the azimuth resolution:	6
1.3 Speckle	7
1.4 Polarimetry	7
1.5 The PolSF ALOS-2 scene	9
2 Complex-valued neural networks	11
2.1 Why keep the complex domain	11
2.2 Differentiating a real loss of complex parameters	11
2.3 Complex layers	12
3 Contrastive self-supervised learning	14
3.1 Self-supervised pre-training	14
3.2 Contrastive learning and the InfoNCE loss	14
3.3 A similarity for complex embeddings	15
4 Data pipeline, spatial split and augmentations	17
4.1 From the SLC product to network inputs	17
4.2 Pre-training regions	17
4.3 A deterministic spatial split of the labelled area	18
4.4 Contrastive augmentations for SAR imaging	19
5 A complex-valued SegFormer	22
5.1 Why SegFormer	22
5.2 Encoder	22
5.3 Decoder and segmentation head	25
5.4 Contrastive heads	25
5.5 Size of the networks	25
6 Training objectives	27
6.1 Multi-stage contrastive learning	27
6.2 Softmax weights regularised towards uniformity	27
6.3 Stage weights as learnable temperatures	28
6.4 Supervised objective	30

7	Experimental protocol	32
7.1	Hardware	32
7.2	Pre-training	32
7.3	Supervised training and fine-tuning	33
7.4	Metrics	33
7.5	Experiments	34
8	Results	36
8.1	Pre-training	36
8.2	Transfer to segmentation	38
8.3	Label efficiency	39
8.4	Relation to published results	41
9	Discussion	42
9.1	Interpretation of the results	42
9.2	Limitations	42
10	Conclusion	44
A	Definition of the augmentations	45
A.1	Random hole	45
A.2	Intensity	45
A.3	Speckle	45
A.4	Elastic deformation	46
A.5	Fourier masking	46
A.6	Rotation and flip	47

List of Figures

1.1	SAR acquisition geometry	2
1.2	Scattering mechanisms	8
1.3	The ALOS-2 San Francisco scene	9
3.1	The SimCLR framework	15
4.1	Mosaic split of the labelled area	18
4.2	Augmentations on a real patch	21
5.1	SegFormer architecture	22
5.2	Encoder stage and decoder branch	23
5.3	Efficient self-attention	24
5.4	Contrastive heads	25
6.1	Focal loss	31
8.1	Pre-training loss	36
8.2	Stage weights during pre-training	37
8.3	Temperatures of LearnableTemp	38
8.4	Convergence of the segmentation training	39
8.5	Label efficiency	40
8.6	Predictions over the labelled area	41

List of Tables

1.1	Class distribution of the labelled area	10
4.1	Pre-training regions	18
4.2	Class balance of the three subsets	19
5.1	Encoder configuration	24
5.2	Number of parameters	26
7.1	Pre-training hyperparameters	33
7.2	Supervised hyperparameters	33
8.1	Test results	38
8.2	Test IoU per class	38
8.3	Test mIoU per share of training patches	40

Chapter 1

Synthetic aperture radar and polarimetry

This chapter introduces some physical concepts around SAR imaging, they mostly help understanding what information lies in the complex nature of the data we fed to our model. This also partly explains why our architecture has to bear complex-valued parameters.

1.1 A coherent measurement

A radar emits an electromagnetic wave of frequency denoted by f_0 and records the echo returned by the ground. A target located at a distance R returns the signal after the round-trip delay $\tau = 2R/c$. For a signal (represented as a simple sinusoid) $A \cos(2\pi f_0 t)$, the echo is:

$$s_r(t) = A \cos(2\pi f_0(t - \tau)) = A \cos(2\pi f_0 t + \varphi), \quad \varphi = -2\pi f_0 \tau = -\frac{4\pi R}{\lambda}, \quad (1.1)$$

where $\lambda = c/f_0$ is the wavelength. The phase of the echo therefore encodes the distance to the target, modulo $\lambda/2$.

A single real-valued recording of s_r cannot recover φ without ambiguity, since $\cos(x) = \cos(-x)$ and $\cos(x + 2\pi) = \cos(x)$. The receiver therefore measures two components. Expanding

$$s_r(t) = A \cos \varphi \cos(2\pi f_0 t) - A \sin \varphi \sin(2\pi f_0 t), \quad (1.2)$$

multiplying by the reference $\cos(2\pi f_0 t)$, respectively by $-\sin(2\pi f_0 t)$, and averaging over a duration much longer than the carrier period $1/f_0$ but shorter than the time scale over which φ varies, keeps the in-phase and quadrature components

$$I = \frac{A}{2} \cos \varphi, \quad Q = \frac{A}{2} \sin \varphi, \quad I + jQ = \frac{A}{2} e^{j\varphi}, \quad (1.3)$$

since $\langle \cos^2 \rangle = \langle \sin^2 \rangle = \frac{1}{2}$ and $\langle \cos \cdot \sin \rangle = 0$ over such a window. Both the amplitude and the phase are then known, $\varphi = \text{atan2}(Q, I)$. This I/Q demodulation is why SAR data is complex-valued from the very first processing step.

1.2 The "synthetic aperture" and radar imaging basics

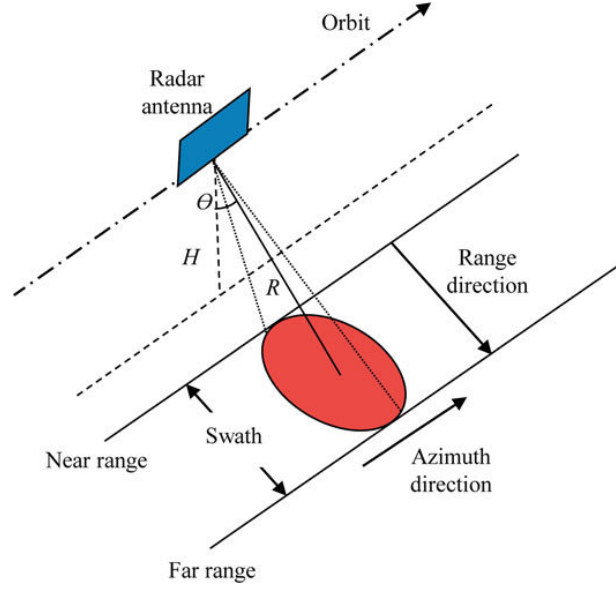


Figure 1.1: SAR acquisition geometry: the sensor moves along the azimuth direction at height H and looks sideways with look angle θ ; R is the slant range to the illuminated area.

Two resolutions characterise an imaging radar: in range and in azimuth. Across the flight direction, two targets are separated in range only if their echoes do not overlap in time. Radars transmit long frequency pulses (chirps) of bandwidth B and compress them on reception by matched filtering, which gives a slant-range resolution

$$\delta_r = \frac{c}{2B}, \quad (1.4)$$

of the order of a few metres for bandwidths of tens of megahertz.

1.2.1 Zoom on the notion of range resolution:

A derivation of this result and its interpretation are not straightforward; it is therefore useful, before getting further into this report, to gain some intuition on how a SAR signal is designed and what constraints it must abide by. The following analysis tries to convey intuition on mathematical tools that enable us to distinguish distant targets while receiving only echoes of a known emitted signal. Let us insist on the fact that what follows rules out any noise assumption which adds some more complexity that does not contribute in building intuition on this matter.

As stated previously, a radar measures the slant range r of a target through the round-trip time $\tau = 2r/c$. If two targets are located at r_1 and r_2 , the delay perceived between the receptions of their echoes is $\delta\tau = 2(r_1 - r_2)/c = 2\delta_r/c$. The most obvious way to tell the two echoes apart is to require that they do not overlap, i.e. that $\delta\tau$ be larger than the pulse duration T . With this in mind we could be tempted to say that to get a smaller and thus better δ_r we need to get a shorter the pulse; however, a short pulse at a single frequency would not carry enough energy to reach long distances. Let us imagine emitting a simple sinusoid during a very short time T ;

the signal can be formalised as:

$$s_T(t) = A \operatorname{rect}\left(\frac{t}{T}\right) e^{j2\pi f_0 t}. \quad (1.5)$$

Ignoring the attenuation along the path, the echo received with a delay τ is $s_T(t - \tau)$, and its energy is that of the pulse:

$$E = \int_{-\infty}^{+\infty} |s_T(t - \tau)|^2 dt = |A|^2 T. \quad (1.6)$$

The echo must not be buried in the noise, so its energy has to remain above a given level: with $T \rightarrow 0$, the amplitude would have to grow to infinity, which is of course impossible. To make it intuitive again, one must think that a sensor both receives the echoed version of the original signal and noise that behaves as a random walk (not necessarily centered, but the key thing is that statistically a sum of its realisations has a standard deviation proportional to the square root of the number of measurements denoted by N). The signal behaves coherently in time on a duration of length T (which indeed is shared by its echo): on a long period of time the noise adds up to the values measured by the sensor grow slowly while the original signal, correlated with the matched filter (computing its autocorrelation for a wide range of plausible delays) adds positive values and grows like the number of measurements so faster than the noise. Hearing an alarm when in the middle of a crowd is easier when it lasts long, a short "bip" would go completely unnoticed whatever its amplitude !

As we will increasingly use the notions of duration and bandwidth, let us specify what we actually mean by them. For the simple pulse designed above, the duration is a fixed window T , delimited by sharp edges. In reality, no emitter can produce such edges: a real pulse rises and decays smoothly, and its duration can only be characterised by a statistical quantity computed in the time domain, which measures how the energy of the pulse is spread in time; the bandwidth is its analogue in the frequency domain. Since $|s(t)|^2$ and $|S(f)|^2$ both integrate to E , once divided by E they can be read as probability densities, in time and in frequency, whose standard deviations σ_t and σ_f define the duration and the bandwidth of the pulse:

$$\sigma_t^2 = \frac{1}{E} \int (t - \bar{t})^2 |s(t)|^2 dt, \quad \sigma_f^2 = \frac{1}{E} \int (f - \bar{f})^2 |S(f)|^2 df, \quad (1.7)$$

where $\bar{t}$ and $\bar{f}$ are the corresponding means. Centering on $\bar{f}$ removes the carrier f_0 , which says nothing about the spread of the spectrum. Those are not to be mixed up with standard deviations of random variables: $s(t)$ and its Fourier transform are perfectly deterministic.

$$S_T(f) = \mathcal{F}(s_T)(f) = AT \operatorname{sinc}(\pi T(f - f_0)), \quad \operatorname{sinc}(x) = \frac{\sin x}{x}, \quad (1.8)$$

whose first zeros are reached for $\pi T(f - f_0) = \pm\pi$ the "bell" that carries most of the energy, spans $f_0 \pm 1/T$ the halfwidth is denoted B as the bandwidth.

Any pulse of fixed amplitude shares this link between the energy it carries and its length, and at first glance the problem seems unsolvable: the energy grows with the duration of the pulse while the resolution degrades with it. While it seems a dead-end, we must outline the fact that two overlapping signals are not necessarily undistinguishable. In reality, radio stations emit many simultaneous signals at different frequencies whose echoes are all perfectly

distinguishable. The autocorrelation function allows to distinguish from two simultaneous but non similar signals:

$$\left| \int s(t - \tau_1) s^*(t - \tau_2) dt \right| = |R(\delta\tau)|, \quad R(\tau) = \int s(t) s^*(t - \tau) dt, \quad (1.9)$$

where R is the autocorrelation of the pulse s .

By the Cauchy–Schwarz inequality, $|R(\delta\tau)| \leq R(0) = E$, with equality only if the two echoes are proportional, in which case no processing can separate them. The two echoes can thus be told apart as soon as $|R(\delta\tau)| < E$, i.e. as soon as they are nearly orthogonal, the width of $|R|$ being some notion of resolution; we rather consider the ratio $\rho = \frac{|R|}{E}$. In practice a matched filter correlates the received signal with a copy of the emitted pulse, so that each echo produces a peak $R(t - \tau_i)$.

For our simple pulse, a direct computation gives $|R(\delta\tau)| = |A|^2(T - |\delta\tau|)$ for $|\delta\tau| < T$ and 0 beyond: a triangle, which only vanishes when the two echoes stop overlapping. For this pulse, and for this kind of pulse only, the non-overlap criterion $\delta\tau \gtrsim T$ is indeed the right one. If this type of pulse is to be used the emitter will face a quandary in deciding which of T or the resolution he is to sacrifice.

But obviously such simple pulse is not the only type of signal at our disposal, we can also pick signals containing a wider range of frequencies. As a matter of fact frequencies will play a crucial role because using the duration of the signal to tune the resolution affects the power of the signal; we are about to show that there is no need to tune the duration to get a signal producing distinguishable echoes: a richer pulse in the frequency domain will also do perfectly fine without sacrificing anything.

First, let us write R in the frequency domain. By Parseval's/Plancherel formula applied to $s(t)$ and $s(t - \tau)$, whose Fourier transforms are $S(f)$ and $S(f) e^{-j2\pi f\tau}$, we get the Wiener–Khinchin relation:

$$R(\tau) = \int |S(f)|^2 e^{j2\pi f\tau} df. \quad (1.10)$$

The autocorrelation is the inverse Fourier transform of the energy spectrum. This entails that the resolution (module of the autocorrelation function) is only dependent on $|S|^2$ and also (for more simplicity imagine a spectrum compactly supported): $R(\tau)$ is just a sum of unit complex numbers over the range of all the frequencies contained in the signal, weighted by the power carried by each frequency. At $\tau = 0$ all these unit numbers add up to E , as τ grows, the unit complex associated to f turns by an angle $2\pi f\tau$, high frequencies rotate faster and two frequencies f and f' end up with an angle $2\pi(f - f')\tau$ between them. The energy of the signal itself is spread over a characteristic range of order $B = \sigma_f$ (this standard deviation was introduced above, for now it is enough to think of it as the only finite quantity measuring the "spread" of the spectrum). With this in mind, we know that the angles of the terms of this integral spread over about $2\pi B\tau$: intuitively, if this spread remains small, all terms in the integral will point in the same direction, leaving an integral close to E , because, again Parseval gives us:

$$E = \int_{-\infty}^{+\infty} |S(f)|^2 df \quad (1.11)$$

In the case of our simple pulse, the spectrum is

$$S_T(f) = AT \operatorname{sinc}(\pi T(f - f_0)), \quad \operatorname{sinc}(x) = \frac{\sin x}{x}, \quad (1.12)$$

whose first zeros are reached for $\pi T(f - f_0) = \pm\pi$ the central "bell" spans $f_0 \pm 1/T$, and we denote its half-width $1/T$ as the bandwidth B of the pulse. This does not change our concern at all: augmenting the resolution calls for a larger bandwidth, which means a smaller duration and thus a weaker signal. But now recall the simple pulse we designed: it contains only a single frequency; here is a simple way to increase the bandwidth of our emitted signal.

While not affecting the duration at all (and also its energy $|A|^2 T$), let us therefore keep the envelope and add a quadratic phase:

$$s(t) = A \operatorname{rect}\left(\frac{t}{T}\right) e^{j(2\pi f_0 t + \pi K t^2)}. \quad (1.13)$$

Its instantaneous frequency, i.e. the derivative of its phase divided by 2π , is $f_0 + Kt$: during the pulse, the frequency sweeps linearly from $f_0 - KT/2$ to $f_0 + KT/2$. Its bandwidth is now:

$$B = |K| T,^1 \quad (1.14)$$

set by the sweep rate K and no longer by the duration of the pulse; a long pulse swept fast has as wide a band as we wanted. Two signals delayed by $\delta\tau$ overlap in time, but at every instant their frequencies differ, and the radio analogy tells us why this is enough. The echo of a target located at delay τ_i is the emitted chirp delayed by τ_i , so that its instantaneous frequency at time t is $f_0 + K(t - \tau_i)$. Two echoes delayed by $\delta\tau = \tau_2 - \tau_1 < T$ overlap in time (over a common interval, the received signal is the sum of both). Yet at every instant of this interval, their instantaneous frequencies differ by the same amount $|K| \delta\tau$, the two echoes overlap in time without resembling each other. This becomes explicit by multiplying the received signal by the conjugate of the emitted pulse:

$$s(t - \tau_i) e^{-j(2\pi f_0 t + \pi K t^2)} = A \operatorname{rect}\left(\frac{t - \tau_i}{T}\right) e^{-j2\pi K \tau_i t} \times \text{constant}, \quad (1.15)$$

that is, a plain sinusoid of frequency $-K\tau_i$ also lasting T ! We know from our first computation that a sinusoid lasting T occupies a band of half-width $1/T$, so the two echoes can be clearly distinguished as soon as their frequencies differ by more than this width:

$$|K| \delta\tau \geq \frac{1}{T} \iff \delta\tau \geq \frac{1}{|K|T} = \frac{1}{B} \iff \delta_r \geq \frac{c}{2B}. \quad (1.16)$$

The resolution finally depends on the bandwidth alone, the duration itself serves only the purpose of setting the energy carried by the pulse. The matched filter turns each long echo into a peak of width $1/B$.

¹For a Gaussian envelope the computation is exact and gives $\sigma_f^2 = 1/(16\pi^2 \sigma_t^2) + K^2 \sigma_t^2$: the first term is the bandwidth the pulse would have without modulation, the second one the spread of its instantaneous frequency during the pulse.

1.2.2 Zoom on the azimuth resolution:

Until now we described the range resolution as we would have for any fixed antenna. Imagery from a fixed antenna has considerable limitations, consider an antenna as being a thin hole on which one projects a laser sight (electromagnetic wave), Huygens and Fresnel proved that light passing through the hole was diffracting and forming interference patterns on the next wall on which light is projected. They also proved that given D the diameter of the hole, θ the angle of the diffraction cone and λ the wavelength of the laser, one has the following equality:

$$\sin(\theta) = \frac{\lambda}{D} \approx \theta \text{ for small angles}$$

This formula validates most of our intuitions, for instance a laser having a small wavelength produces a concentrated beam that naturally diffracts very little, which is the exact contrary of what a radar emits. It also tells us that for a large antenna, one closes the cone and gets a concentrated signal that projects most of its energy on a small target on the ground. Most of its frequencies comprised in its spectrum are then echoed by this small target (the size of the antenna acts as a spatial filter) that is consequently well described by the radar. On the opposite, a small antenna sweeps a large area but is not precise at all. This is problematic since we want a way to precisely describe the ground on a large area, large antennas are difficult to build and carry while small antennas are not accurate enough. That is where SAR imaging comes in: carry a small antenna onto a plane and sweep a large area flying over the same targets and then aggregate the echoes produced by the Doppler effect to get an exhaustive description of all targets on the ground. The azimuth is the missing component to describe most of the inner working of SAR imaging.

In azimuth, along the flight direction, a real antenna of length D has a beam of angular width about λ/D and would only separate targets $\lambda r_0/D$ apart, r_0 being the distance above the ground. We said the synthetic aperture overcomes this limit by exploiting the motion of the radar, the mechanism is then similar to what we described in the previous subsection. As the sensor flies at speed v past a target, its distance to the target evolves as

$$r(t) = \sqrt{r_0^2 + v^2 t^2} \approx r_0 + \frac{v^2 t^2}{2r_0}, \text{ through a Taylor expansion} \quad (1.17)$$

so that, through (1.1), the phase of the successive echoes is quadratic in time: the target produces a chirp, of rate $K_a = -2v^2/(\lambda r_0)$, created by the geometry of the acquisition rather than by the transmitter as we did earlier. This is precisely the Doppler effect of a target first approaching and then receding. The target remains in the beam during $T_a = \lambda r_0/(Dv)$, the time needed to fly over the synthetic aperture $L_{sa} = 2r_0 \sin(\frac{\theta}{2}) \approx \lambda r_0/D$, and the Doppler band swept meanwhile is

$$B_a = |K_a| T_a = \frac{2v}{D}. \quad (1.18)$$

The argument of the previous section then applies unchanged: the illumination time only sets the collected energy, while the resolution in time is $1/B_a$, i.e. in distance

$$\delta_a = \frac{v}{B_a} = \frac{D}{2}. \quad (1.19)$$

This stripmap result is independent of the range and of the wavelength [24]: a farther target

is illuminated longer but with a slower chirp, and both effects cancel out. It also explains the paradox that a smaller antenna gives a finer resolution: its wider beam keeps the target illuminated longer, hence sweeps a wider Doppler band. The output of this focusing step is called a single-look complex (SLC) image: along its trajectory in the azimuth direction the radar emits and measures the echoes coming back at different instants which indicates targets located at different distances. Those measurements are then stored in a 2D grid, each sampled echo at instant t ($t = 0$ being the instant when the pulse is emitted) filling a coefficient on the row associated to the current position in azimuth. Figure 1.1 summarises the acquisition geometry.

1.3 Speckle

A resolution cell of a few metres contains many elementary scatterers: leaves, stones, parts of buildings. Each pixel's value is the coherent sum of their contributions:

$$z = \sum_{k=1}^K a_k e^{j\varphi_k}, \quad (1.20)$$

where each phase φ_k depends, through (1.1), on the position of the scatterer at the scale of the wavelength. For a distributed target, these phases are unpredictable and can be modelled as independent and uniformly distributed. When K is large, the central limit theorem makes z a complex random variable whose real and imaginary part can both be modeled by a Gaussian distribution of variance $\sigma^2/2$, σ^2 being called the reflectivity of the surface [9]. Its amplitude $|z|$ follows a Rayleigh distribution and its intensity $|z|^2$ an exponential distribution of mean σ^2 , whose standard deviation equals its mean. In other words, a SAR image of a perfectly homogeneous field is not homogeneous: it is the reflectivity multiplied by a random factor,

$$|z|^2 = \sigma^2 n, \quad n \sim \text{Exp}(1). \quad (1.21)$$

This multiplicative form of noise is what is called "speckle" in literature. Averaging L independent looks reduces it: the averaged intensity follows a Gamma distribution of mean σ^2 and variance σ^4/L . Speckle has two consequences for learning. The information carried by a distributed target lies in the statistics of a neighbourhood of pixels rather than in any single pixel, which favours networks that aggregate spatial context. And speckle is a natural source of variability between two observations of the same ground, which we use in Section 4.4 to generate contrastive views.

The same argument shows that the absolute phase of a pixel of a distributed target is uniformly distributed and carries no information on its own. The information lies in phase differences: between polarimetric channels of the same pixel, as described below, or between acquisitions in interferometry. This observation motivates the choice of a phase-invariant contrastive similarity in Section 3.3.

1.4 Polarimetry

A polarimetric radar transmits alternately with a horizontally (H) and a vertically (V) polarised antenna and receives both polarisations simultaneously. For every pixel it therefore measures

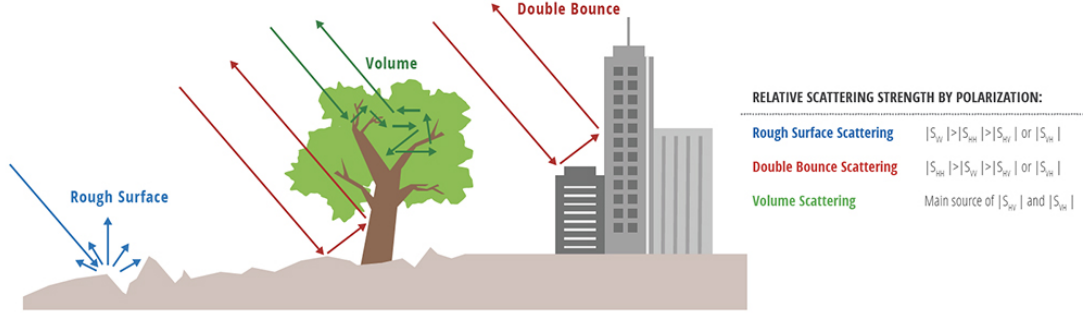


Figure 1.2: Dominant scattering mechanisms as a function of the ground cover and of the wavelength: surface scattering, volume scattering in vegetation, and double bounce on structures. Longer wavelengths, such as the L band used here, penetrate deeper into the canopy. Credits: NASA SAR handbook [25].

the 2×2 complex scattering matrix, or Sinclair matrix,

$$\mathbf{S} = \begin{pmatrix} S_{HH} & S_{HV} \\ S_{VH} & S_{VV} \end{pmatrix}, \quad (1.22)$$

where S_{pq} is the response received in polarisation p to a transmission in polarisation q . For a monostatic radar, where the same antenna transmits and receives, reciprocity imposes $S_{HV} = S_{VH}$; in measured data the two cross-polarised channels agree up to noise and residual calibration errors.

The scattering matrix is usually vectorised in one of two bases [17]. The lexicographic basis gives $\mathbf{k}_L = (S_{HH}, \sqrt{2} S_{HV}, S_{VV})^T$, and the Pauli basis gives

$$\mathbf{k}_P = \frac{1}{\sqrt{2}} (S_{HH} + S_{VV}, S_{HH} - S_{VV}, 2S_{HV})^T. \quad (1.23)$$

The Pauli components have a direct physical reading [5]. The first one dominates for single-bounce (odd) scattering, as on bare soil or water surface; the second for double-bounce (even), as on corners formed by a wall and the ground, typical of buildings; the third, grows with volume scattering, as in forest. PolSAR images are often displayed in Pauli colours, red for $|S_{HH} - S_{VV}|^2$, green for $|S_{HV}|^2$ and blue for $|S_{HH} + S_{VV}|^2$, as in Figure 1.3. Figure 1.2 illustrates the dependence of these mechanisms on the type of ground cover and on the wavelength.

The network of this work receives, for every pixel, the three-component complex vector

$$\mathbf{x} = \left(S_{HH}, \frac{1}{2}(S_{HV} + S_{VH}), S_{VV} \right)^T \in \mathbb{C}^3, \quad (1.24)$$

that is, the lexicographic vector in which the two cross-polarised channels are averaged, which enforces reciprocity and reduces their noise, and in which the cross-polarised term is not multiplied by $\sqrt{2}$. The Pauli vector is an invertible linear transform of $\mathbf{x}$; since the first layer of a complex-valued network applies a learnable complex linear map to the channels, it can represent the change of basis exactly, and the choice of basis does not restrict what the network can learn.

1.5 The PolSF ALOS-2 scene

The data of this work is the San Francisco acquisition of the PALSAR-2 sensor on board the Japanese ALOS-2 satellite, distributed by the IETR [13] and annotated in the PolSF benchmark [20]. According to the metadata of the product, it is an L-band acquisition, with a wavelength of 24.2 cm, in the High-sensitive mode with full (quad) polarimetry, acquired on 24 March 2015 and provided as a level 1.1 single-look complex product in the CEOS format [14]: one volume directory file, one leader file holding, among other things, the radiometric calibration factor, one trailer file and one image file per polarisation, HH, HV, VH and VV. The scene measures 22,608 lines by 8,080 samples and covers the San Francisco peninsula, the northern part of the bay and the surrounding hills (Figure 1.3).

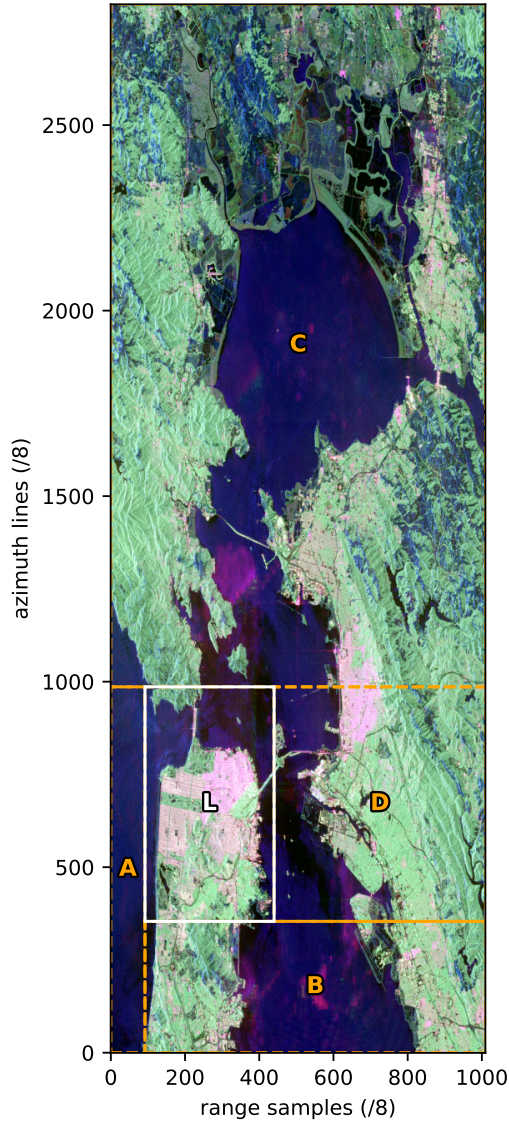


Figure 1.3: The ALOS-2 San Francisco scene in Pauli colours (red: double bounce, green: volume, blue: surface), averaged over 8×8 pixels for display. The white rectangle L is the labelled area of PolSF, used for the supervised task only. The four dashed rectangles A to D tile the rest of the scene and provide the unlabelled patches of the contrastive pre-training (Section 4.2).

Table 1.1: Class distribution in the labelled area of PolSF, over the part covered by the 79×43 grid of 64×64 patches used in this work.

Label	Class	Pixels	Share of labelled pixels
1	Mountain	354,280	4.44%
2	Water	4,648,961	58.29%
3	Vegetation	459,701	5.76%
4	High-density urban	1,314,858	16.49%
5	Low-density urban	947,418	11.88%
6	Developed	250,403	3.14%
0	Unlabelled	5,938,491	(42.7% of all pixels)

Labels. The PolSF annotations [20] cover a rectangle of 5,056 lines by 2,784 samples, located between lines 2,832 and 7,888 and samples 736 and 3,520 of the scene, ($\approx 7.7\%$ of the acquisition). Each pixel of this rectangle belongs to one of six classes, mountain, water, vegetation, high-density urban, low-density urban and developed, or is left unlabelled (class 0). See Table 1.1 to appreciate the distribution of the classes. It must be kept in mind that the classes are strongly imbalanced: water accounts for more than half of the labelled pixels, while mountain and developed areas each account for less than 5%. And finally, 42.7% of the pixels of the labelled rectangle are unlabelled (and thus kept out of metrics computations), mostly along class boundaries and in large areas that were not annotated (Figure 4.1a); as a result, 29% of the 64×64 patches used in this work contain no labelled pixel at all, this is a notable issue for supervised training as we will later see.

The part of the scene outside the labelled rectangle, 92.3% of the acquisition, carries no annotation. It represents around 41000 64×64 patches which constitutes a natural source of data for contrastive learning while being quite scarce. Contrastive pretrainings are usually performed on millions of images: in [4] (SimCLR) the authors used the dataset described in [28], ILSVRC2012 that consisted in 1.2 million images, far greater than what we dispose of.

Chapter 2

Complex-valued neural networks

A PolSAR image is a field of complex vectors. This chapter explains why this work keeps the whole network complex-valued, how such a network can be trained by gradient descent although its loss is not a holomorphic function, and which complex layers are used in the rest of the report.

2.1 Why keep the complex domain

The simplest way to feed SAR data to a standard network is to keep only the amplitudes, or the intensities, of the channels. This discards the phase differences between polarimetric channels, which Section 1.4 showed to be physically meaningful. A second option is to split every complex channel into its real and imaginary parts and to process the 2C resulting real channels with a real-valued network. Nothing is lost, but the algebraic structure of the data is not used: multiplying a complex number by $e^{j\theta}$ is a rotation of its real and imaginary parts, and a real-valued network has to learn from data that such rotations are special.

A complex-valued neural network (CVNN) keeps this structure in its parameters [3, 29]. A complex weight $w = a + jb$ acting on $z = x + jy$ computes

$$wz = (ax - by) + j(bx + ay), \quad \text{that is} \quad \begin{pmatrix} \Re(wz) \\ \Im(wz) \end{pmatrix} = \begin{pmatrix} a & -b \\ b & a \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} : \quad (2.1)$$

a scaling combined with a rotation, two degrees of freedom instead of the four of a general 2×2 real matrix. Complex layers therefore constrain how the real and imaginary parts can interact, in a way that matches how phase behaves physically. Whether this inductive bias improves accuracy is an empirical question, studied for PolSAR by Barrachina et al. [2]; in this work, it is a design choice shared by all the compared models.

2.2 Differentiating a real loss of complex parameters

In complex analysis, a function $f : \mathbb{C} \rightarrow \mathbb{C}$ is differentiable at z when the limit of $(f(z + h) - f(z))/h$ exists as $h \rightarrow 0$ in $\mathbb{C}$; functions that are differentiable everywhere are holomorphic, or entire. This notion is too strong for learning, for two reasons. Liouville's theorem states that a bounded entire function is constant, so the bounded non-linearities used in neural networks cannot be holomorphic. More fundamentally, the loss $\mathcal{L}$ minimised during training is real-valued, and a real-valued function of a complex variable satisfies the Cauchy–Riemann equations only if it is constant: no useful loss is holomorphic.

The solution is to consider $\mathcal{L}$ as a function of the two real variables $x = \Re z$ and $y = \Im z$, which is differentiable in the ordinary real sense, and to express its derivatives with the Wirtinger

operators [16]

$$\frac{\partial f}{\partial z} = \frac{1}{2} \left(\frac{\partial f}{\partial x} - j \frac{\partial f}{\partial y} \right), \quad \frac{\partial f}{\partial \bar{z}} = \frac{1}{2} \left(\frac{\partial f}{\partial x} + j \frac{\partial f}{\partial y} \right). \quad (2.2)$$

For a holomorphic f , $\partial f / \partial \bar{z} = 0$ and $\partial f / \partial z$ is the usual complex derivative, so the Wirtinger calculus extends complex differentiation. For a real-valued $\mathcal{L}$, $\partial \mathcal{L} / \partial \bar{z} = \overline{\partial \mathcal{L} / \partial z}$, and the direction of steepest ascent of $\mathcal{L}$ in the complex plane is $2 \partial \mathcal{L} / \partial \bar{z}$. Gradient descent on a complex parameter therefore reads

$$z \longleftarrow z - \eta \cdot 2 \frac{\partial \mathcal{L}}{\partial \bar{z}}, \quad (2.3)$$

which is exactly the real gradient descent on (x, y) written in complex notation. Compositions are handled by the chain rule

$$\frac{\partial(f \circ g)}{\partial z} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial z} + \frac{\partial f}{\partial \bar{g}} \frac{\partial \bar{g}}{\partial z}, \quad (2.4)$$

which reduces to the usual rule when f is holomorphic. PyTorch implements this convention in its automatic differentiation: for a real loss it returns the conjugate Wirtinger derivative, so that standard optimisers such as AdamW apply to complex parameters without modification.

2.3 Complex layers

The network of this work is built with PyTorch’s native complex tensors and the `torchcvnn` library [8], developed at CentraleSupélec and at the SONDRALABORATORY, which provides complex versions of the usual layers, SAR datasets and transforms. The layers used are the following.

Linear layers and convolutions. They are the usual operations with complex weights and biases. A complex convolution of $z = x + jy$ by $W = A + jB$ amounts to four real convolutions, $W * z = (A * x - B * y) + j(B * x + A * y)$.

Activations. Two activations are used. The `modReLU` of Arjovsky et al. [1] acts on the modulus and preserves the phase,

$$\text{modReLU}(z) = \text{ReLU}(|z| + b) e^{j \arg z}, \quad (2.5)$$

where b is a learnable real threshold, one per layer in `torchcvnn`: a negative b switches off the components whose modulus is below $|b|$, whatever their phase. The `CGELU` applies the GELU non-linearity [11] separately to the real and imaginary parts, $\text{CGELU}(z) = \text{GELU}(\Re z) + j \text{GELU}(\Im z)$; it is used inside the feed-forward blocks of the transformer, where `SegFormer` uses a GELU.

Normalisation. The complex batch normalisation of Trabelsi et al. [29] treats every complex feature as a two-dimensional real random vector. For each channel it subtracts the mean and multiplies by the inverse square root of the 2×2 covariance matrix of the real and imaginary parts, which whitens the distribution, rather than dividing by a standard deviation. A learnable 2×2 real matrix and a complex bias then restore the scale. At inference, the mean

and the covariance are replaced by running averages estimated during training, so that the output of a sample does not depend on the other samples of its batch.

Note 2.1

Notes and contribution to torchcvnn (0.9.4): Another popular normalisation layer is the `LayerNorm` in Pytorch which is also implemented in `torchcvnn`. It performs a normalisation at a single sample scale, it computes both mean and standard deviation across all values, subtracts by the mean and divides by the standard deviation and adds learnable parameters either for each element in the tensor adding more parameters than the batch normalisation mentioned earlier. The layernorm in our case was applied over the channels, meaning statistics are computed for each pixel across its channels, which seemed necessary to preserve the polarization structure of our data. During our experiments, using layernorm proved to be less efficient and less stable than the batchnorm which was kept wherever the original SegFormer uses layernorms for the results exhibited in this document, a choice discussed in Section 9.2. However we need to outline two different fixes made to `torchcvnn` made during our recent experiments. The `LayerNorm` implemented in the library was permuting the batch dimension with the raveled dimension and computing statistics batch-wise as a `batchnorm` would, this is a critical issue. A second issue was that the `BatchNorm` as implemented in the library was not keeping the history of running statistics out of the autograd computation graph leading to memory leaks and OOM issues, it was fixed with a simple `.detach()`.

Attention. Transformers compute attention weights with a softmax, which requires real scores. Following Eilers and Jiang [7], the complex multi-head attention of `torchcvnn` uses the real part of the Hermitian product between queries and keys,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{\Re(QK^H)}{\sqrt{d_k}}\right) V, \quad (2.6)$$

where d_k is the dimension of a head. Since $\Re(q^H k) = \sum_i |q_i| |k_i| \cos(\arg k_i - \arg q_i)$, a key receives a high weight when it is aligned in phase with the query. The attention weights are real and sum to one, and the output is a convex combination of complex values.

Resampling and regularisation. Upsampling interpolates the real and imaginary parts separately, which is the same as interpolating the complex values, since interpolation is linear. Dropout and stochastic depth [12] multiply complex tensors by real random masks.

From complex outputs to class probabilities. The last layer of the segmentation network produces one complex score o_c per class and per pixel. Class probabilities are obtained from their moduli,

$$p_c = \frac{\exp(|o_c|)}{\sum_{c'} \exp(|o_{c'}|)}, \quad (2.7)$$

so that the phase is discarded at the very end of the network only.

Chapter 3

Contrastive self-supervised learning

This chapter introduces contrastive learning, the self-supervised method used to pre-train the encoder, and adapts it to complex-valued embeddings.

3.1 Self-supervised pre-training

Self-supervised learning trains a network on a pretext task whose supervision is derived from the data itself, then reuses the trained network, or part of it, for a downstream task with few labels. Pretext tasks include reconstructing masked parts of the input, predicting the relative position of patches, or, in contrastive learning, recognising two transformations of the same input. The pre-trained weights are then either frozen, with only a small head trained on top (linear probing), or used as the initialisation of a network trained on the labelled data (fine-tuning). This work mainly uses fine-tuning, which is the relevant setting when the goal is the best possible segmentation, and evaluates the pre-training by comparing the fine-tuned network with an identical network trained from a random initialisation.

3.2 Contrastive learning and the InfoNCE loss

Contrastive learning [4, 10, 30] starts from a family of random transformations $\mathcal{T}$, the augmentations, that are assumed to preserve the semantic content of an input. Two augmentations $t, t' \sim \mathcal{T}$ applied to the same input x produce two views $t(x)$ and $t'(x)$, which form a positive pair. An encoder f followed by a small projection head g maps each view to an embedding, $z = g(f(t(x)))$ (Figure 3.1). The network is trained so that the two embeddings of a positive pair are more similar to each other than to the embeddings of the other inputs of the batch, which act as negatives. After training, the projection head is discarded and the encoder f is transferred.

Consider a batch of N inputs. Their $2N$ views give embeddings $z_1, \dots, z_{2N}$, where z_i and z_{i+} denote the two views of the same input. Given a similarity function sim and a temperature $\tau > 0$, the normalised temperature-scaled cross-entropy (NT-Xent) loss of SimCLR is

$$\ell(\tau) = \frac{1}{2N} \sum_{i=1}^{2N} -\log \frac{\exp(\text{sim}(z_i, z_{i+})/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}. \quad (3.1)$$

The contrastive learning framework as exposed in [4] must be seen as a classification problem. For an anchor z_i , the $2N - 1$ other embeddings are candidates, one of which is its

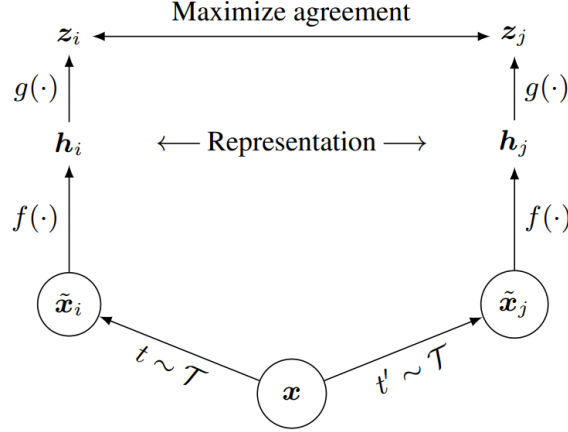


Figure 3.1: The SimCLR framework. Two augmentations t, t' of the same input give two views, encoded by f and projected by g ; the contrastive loss pulls the two embeddings together and pushes them away from the embeddings of the other inputs of the batch. Adapted from Chen et al. [4].

positive. The quantity inside the logarithm is the probability that the categorical model

$$p_\tau(k | i) = \frac{\exp(\text{sim}(z_i, z_k)/\tau)}{\sum_{k' \neq i} \exp(\text{sim}(z_i, z_{k'})/\tau)}, \quad k \neq i, \quad (3.2)$$

assigns to the correct candidate i^+ . NT-Xent is therefore the negative log-likelihood of a $(2N - 1)$ -way classification problem, whose logits are similarities divided by the temperature. van den Oord et al. [30] named this loss InfoNCE and showed that it bounds from below the mutual information between the two views. This probabilistic reading is the basis of the stage weighting proposed in Section 6.3. It is clear to see that the temperature sets both which negatives matter and how strongly the loss drives the encoder.

Why are negative examples needed? One problem with a loss measuring only the similarity between positive examples is called collapsing. The network finds a trivial way to minimize the loss: predict always the same embedding for all examples. The denominator rewards embeddings of different inputs for being dissimilar, which is what prevents this collapse. The number of negatives is set by the batch size: with the batches of $N = 16$ inputs used here (mostly because of hardware constraints), each anchor has 30 negatives, far fewer than the thousands used on natural images, a limitation discussed in Section 9.2.

3.3 A similarity for complex embeddings

For real embeddings, SimCLR uses the cosine similarity $z^T z' / (\|z\| \|z'\|)$. Its complex counterpart is the normalised Hermitian product

$$\gamma(z, z') = \frac{z^H z'}{\|z\| \|z'\|} \in \mathbb{C}, \quad z^H z' = \sum_{d=1}^D \bar{z}_d z'_d, \quad (3.3)$$

which is complex and must be turned into a real score. Two choices are natural: its real part, $\Re \gamma(z, z') \in [-1, 1]$, and its modulus, $|\gamma(z, z')| \in [0, 1]$.

This work uses the modulus,

$$\text{sim}(z, z') = |\gamma(z, z')| = \frac{|z^H z'|}{\|z\| \|z'\|}, \quad (3.4)$$

for the following reason. Multiplying an embedding by a unit complex number leaves the modulus unchanged, $\text{sim}(e^{j\theta} z, z') = \text{sim}(z, e^{j\theta} z') = \text{sim}(z, z')$, whereas it rotates γ and changes its real part. Section 1.3 showed that the absolute phase of SAR data has no physical meaning on its own. A similarity based on the real part would require the two views of a patch to produce embeddings with the same absolute phase, and two patches of the same land cover could be told apart by a quantity that carries no information. The modulus measures only how collinear the two embeddings are in $\mathbb{C}^D$: by the Cauchy–Schwarz inequality, $\text{sim}(z, z') = 1$ if and only if $z' = c z$ for some complex $c \neq 0$.

Remark 3.1. The attention of Equation (2.6) makes the opposite choice and uses the real part of the Hermitian product. The two choices answer different needs: inside the network, phase alignment between features is information worth exploiting; between the final embeddings of two views, it is not.

Chapter 4

Data pipeline, spatial split and augmentations

This chapter describes how the scene of Section 1.5 is turned into training examples: the normalisation of the complex data, the separation between the unlabelled pre-training regions and the labelled area, the split of the labelled area into training, validation and test subsets, and the augmentations that generate contrastive views.

4.1 From the SLC product to network inputs

The four polarisation images are read with the ALOS-2 reader of `torchcvnn`, and every pixel is multiplied by the calibration factor of the leader file, so that the values are calibrated complex amplitudes. The three channels of (1.24) are then formed, giving a tensor in $\mathbb{C}^{3 \times H \times W}$.

Amplitude normalisation. SAR amplitudes span several orders of magnitude, from dark calm water to the bright double bounce of buildings, and their distribution is heavy-tailed. The amplitude of every complex value is therefore compressed logarithmically while its phase is kept:

$$z \mapsto \frac{\log_{10}(\text{clip}(|z|, a_{\min}, a_{\max})) - \log_{10} a_{\min}}{\log_{10} a_{\max} - \log_{10} a_{\min}} e^{j \arg z}, \quad a_{\min} = 0.02, \quad a_{\max} = 40, \quad (4.1)$$

which maps the modulus to $[0, 1]$. This is the `LogAmplitude` transform of `torchcvnn`, applied channel-wise to every patch.

Patches. The images are cut into non-overlapping 64×64 patches on a regular grid, with a stride of 64 pixels. A patch covers 64×64 pixels of the SLC grid, that is, the same number of lines and samples; it is not square on the ground, since the azimuth and range pixel spacings differ.

4.2 Pre-training regions

The contrastive pre-training uses only the part of the scene that is not labelled. The complement of the labelled rectangle is tiled by four rectangles, labelled A to D in Figure 1.3, whose union is exactly the scene minus the labelled area. Each rectangle is cut into patches independently (Table 4.1). The pre-training set contains 40,958 patches, twelve times the 3,397 patches of the labelled area, and none of its pixels belongs to the labelled rectangle: the pre-training never sees the data on which the networks are evaluated.

Table 4.1: Pre-training regions, in SLC coordinates (lines $\times$ samples, end excluded), and number of 64×64 patches.

Region	Lines	Samples	Size	Patches
A	[0, 7888)	[0, 736)	7888×736	1,353
B	[0, 2832)	[736, 8080)	2832×7344	5,016
C	[7888, 22608)	[0, 8080)	14720×8080	28,980
D	[2832, 7888)	[3520, 8080)	5056×4560	5,609
Total pre-training				40,958
L	[2832, 7888)	[736, 3520)	5056×2784	3,397

4.3 A deterministic spatial split of the labelled area

The labelled rectangle is cut into a grid of 79×43 patches, which must be divided into a training set, a validation set used to select the checkpoint, and a test set used once, at the end, to report results.

Drawing patches at random, as is common, has two drawbacks for a single scene. Neighbouring patches are more likely to be much correlated, and a random draw might draw test patches next to training patches, or aggregate them with a pattern that changes with the random seed and adds a strong bias in the predictions. The measured score then depends on the draw, and two configurations evaluated on different draws cannot be compared reliably.

We use instead a fixed lattice. The patch at row i and column j of the grid is assigned the cell

$$c(i, j) = (i + 2j) \bmod 5, \quad (4.2)$$

and the cells are dealt as follows: cell 0 to the test set, cell 1 to the validation set, and cells 2, 3 and 4 to the training set. This sends three fifths of the patches to training and one fifth to each of validation and test (Figure 4.1).

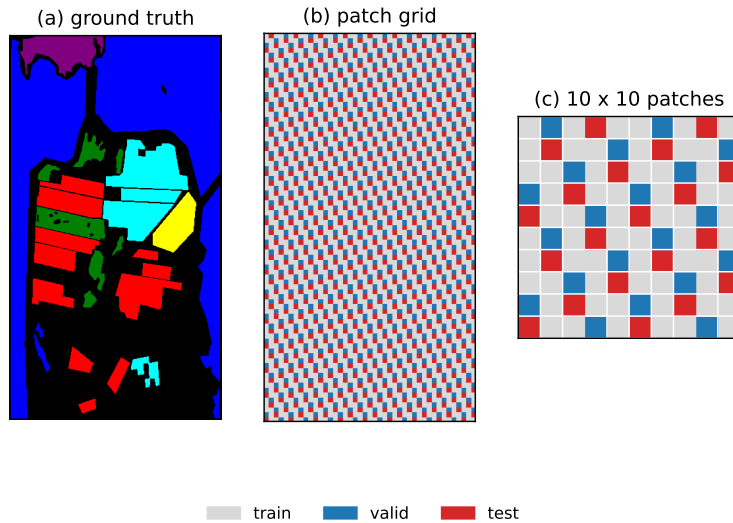


Figure 4.1: The labelled area of PolSF. (a) Ground truth; black pixels are unlabelled / (b) Assignment of the 79×43 patches to the training (grey), validation (blue) and test (red) sets by the lattice (4.2) / (c) Detail of 10×10 patches: no two test patches touch, and every test patch is surrounded by three training patches and one validation patch.

Note that under the assignment (4.2), no two test patches are adjacent, even diagonally, and every test patch that does not lie on the border of the grid has, among its four direct neighbours, exactly one validation patch and three training patches.

Note 4.1

Moving by one patch changes the cell by ± 1 vertically and by ± 2 horizontally, and by $\pm 1 \pm 2 \in \{\pm 1, \pm 3\}$ diagonally, modulo 5. None of these steps is a multiple of 5, so the eight neighbours of a test patch (cell 0) are never in cell 0. The four direct neighbours have cells 1, 4, 2, 3, respectively above, below, right and left: one validation patch and three training patches.

The lattice needs no random seed, so every experiment of this report is evaluated on exactly the same test patches. Another good property of this split is that it also preserves the class balance of the scene: Table 4.2 shows that the three subsets agree within a third of a percentage point on every class.

Table 4.2: Size of the three subsets, and share of each class among their labelled pixels.

Subset	Patches	Mountain	Water	Veget.	HD urban	LD urban	Developed
Train	2,038	4.53%	58.17%	5.76%	16.44%	11.93%	3.17%
Validation	679	4.31%	58.46%	5.68%	16.51%	11.96%	3.09%
Test	680	4.30%	58.48%	5.87%	16.59%	11.66%	3.10%

Finally, the assignment (4.2) standardizes the bias across seeds by removing any possible lucky draw from the test split selection. Also it allows to better evaluate the model's ability to generalize even if in our case this must not be exaggerated since all patches belong to the same scene. Note also that all patches do not overlap, so no pixel belongs to several subsets.

4.4 Contrastive augmentations for SAR imaging

The augmentations define what the pre-trained encoder learns to ignore. In classical imaging, augmentations are typically what we could call "semantic invariances": modifications that do not change the meaning of the data. An image representing a dog still represents a dog whether it is rotated, blurry or wholed in several places. However knowing what are valid invariances for a SAR image is less intuitive since the understanding of a scene is not at our reach. Instead we can try to design augmentations teaching the model what physically realistic degradations do to a SAR image. They must therefore respect three principles:

1. They act on the calibrated complex signal, before the logarithmic normalisation (4.1), so that multiplicative operations keep their physical meaning.
2. They preserve the relative phases between the polarimetric channels and between neighbouring pixels, which carry the polarimetric and spatial information. Augmentations that perturb the phase of each pixel independently are therefore excluded: they would destroy precisely the information the network should learn to use.
3. They should not ask the network to be invariant to transformations that change the physics of the acquisition. A rotation of the patch, for instance, changes the direction from which the scene appears to be illuminated, and radar echoes depend on that

direction. This explains why we always apply rotations as data augmentations only and never to produce positive pairs.

Following these principles, the pipeline uses the transformations below; their precise definitions are given in Appendix A, and Figure 4.2 shows them on a real patch.

- **Random hole:** one or two rectangles, whose sides are 5% to 15% of the patch size, are set to zero in all channels.
- **Intensity:** each channel is multiplied by a real factor drawn uniformly in $[0.95, 1.05]$; the phases are unchanged.
- **Speckle:** the modulus of every pixel of every channel is multiplied by an independent Gamma variable of shape 5 and mean 1 (variance 0.2), and its phase is kept. This simulates a different realisation of the multiplicative fluctuation of Section 1.3.
- **Elastic deformation:** the patch is warped by a smooth random displacement field of amplitude up to 5 pixels; the real and imaginary parts are resampled bilinearly with the same field.
- **Fourier masking:** the two-dimensional spectrum of each channel is multiplied by a structured binary mask made of lines, rings or angular sectors, a central disc of low frequencies being always preserved; each channel receives its own mask.
- **Rotation and flip:** rotations by multiples of 90° and horizontal or vertical flips.

Composition of the two views. Given a patch, the pipeline first applies, with probability $\frac{1}{2}$, a random rotation or flip to the patch itself; both views are derived from the result, so they share their orientation. In agreement with the third principle, the geometric transforms enlarge the pre-training set without asking the network to be invariant to them. The two views are then built with complementary transformations:

$$\text{view 1} = \text{random hole} \rightarrow \text{intensity} \rightarrow \text{speckle} \rightarrow \text{elastic deformation (probability } \frac{1}{4}), \quad (4.3)$$

$$\text{view 2} = \text{random hole} \rightarrow \text{Fourier masking}, \quad (4.4)$$

the random holes being drawn independently for each view. The first view perturbs the image in the spatial and radiometric domain, the second in the spectral domain; to match them, the encoder must rely on the content that survives both kinds of perturbation. Each view is finally normalised by (4.1).

The supervised training of Chapter 7 uses no augmentation, so that the pre-trained and randomly initialised networks see exactly the same labelled data.

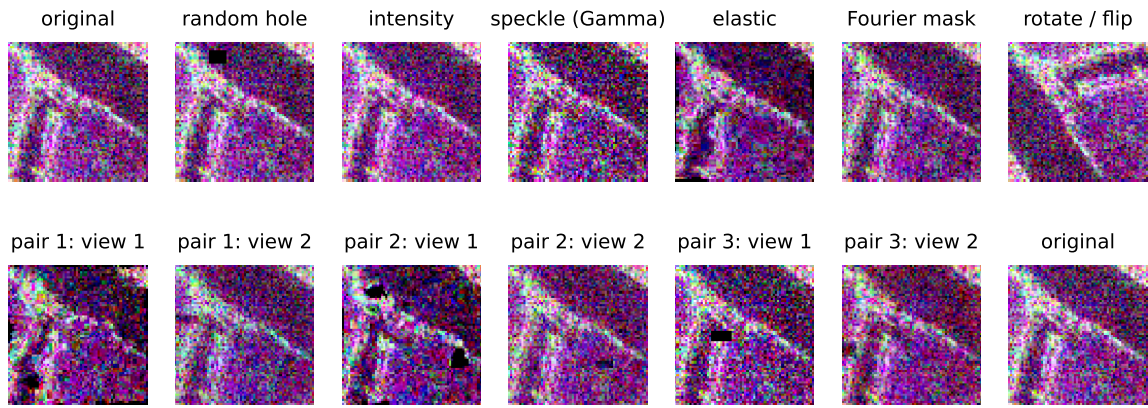


Figure 4.2: Augmentations on a 64×64 patch of region C, displayed as the log-amplitudes of the three channels (red: HH, green: HV, blue: VV) with the same contrast stretch in every panel. Top: each transformation alone; for visibility, the intensity factor range is widened to $[0.7, 1.3]$ and the elastic deformation is always applied. Bottom: three pairs of views produced by the pre-training pipeline, with its actual parameters.

Chapter 5

A complex-valued SegFormer

The segmentation network is a complex-valued re-implementation of SegFormer [31], written for this work on top of torchcvm. This chapter describes its encoder, its decoder, and the contrastive heads added to the encoder for pre-training.

5.1 Why SegFormer

SegFormer combines a hierarchical transformer encoder, the Mix Transformer, with a light decoder made of point-wise layers only (Figure 5.1). Three of its properties suit this work. The encoder produces a pyramid of feature maps at four resolutions, as convolutional segmentation networks do, which is what dense prediction needs. It uses no positional encoding, positional information being provided by depth-wise convolutions, so the same network applies to any input size. And most of its parameters sit in the encoder, which is the part that self-supervised pre-training can improve: in the configuration used here, the encoder holds 87% of the parameters.

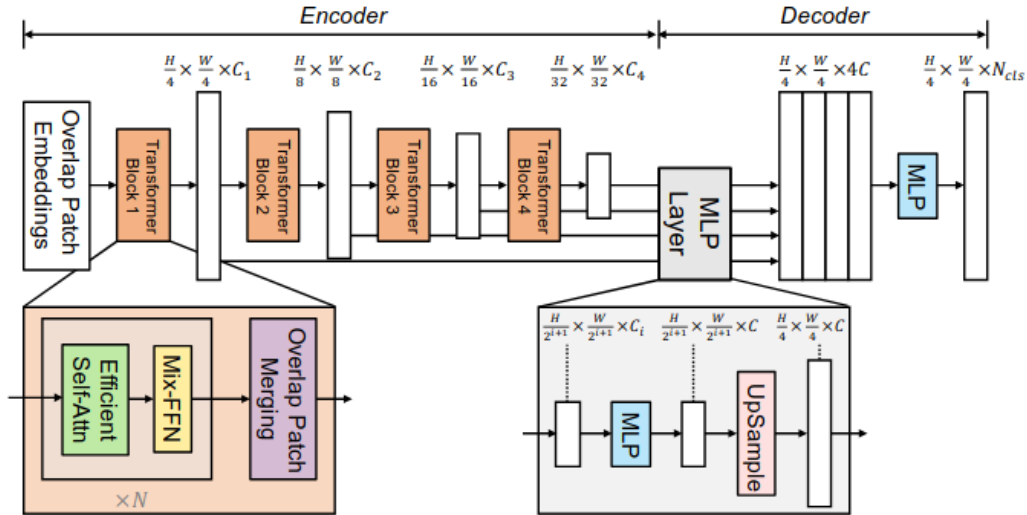


Figure 5.1: SegFormer: a hierarchical encoder of four transformer stages produces feature maps at $1/4$, $1/8$, $1/16$ and $1/32$ of the input resolution; an all-MLP decoder brings them to a common resolution, fuses them and predicts the class of every pixel. Adapted from Xie et al. [31].

5.2 Encoder

The encoder is a sequence of four stages. Each stage starts with an overlapping patch merging, a strided complex convolution followed by a normalisation, which reduces the spatial resolution and increases the number of channels. It then applies two transformer blocks and a final

normalisation (Figure 5.2). A block updates its input X in two residual steps,

$$X \leftarrow X + \text{EfficientAttention}(\text{Norm}(X)), \quad (5.1)$$

$$X \leftarrow X + \text{DropPath}(\text{MixFFN}(\text{Norm}(X))), \quad (5.2)$$

where all operations are complex-valued and Norm is the complex batch normalisation of Section 2.3, which replaces the layer normalisation of SegFormer (Section 9.2). The other normalisations of the network are the same layer.

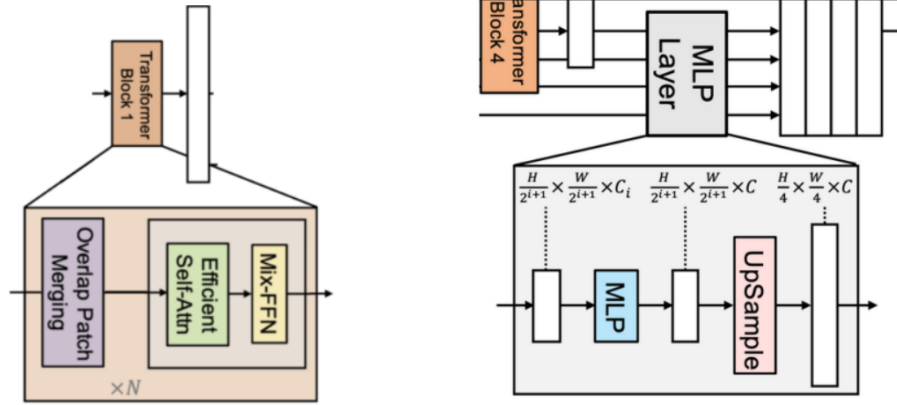


Figure 5.2: Left: an encoder stage, made of an overlapping patch merging followed by N blocks of efficient self-attention and Mix-FFN. Right: one branch of the decoder, which projects a feature map to a common number of channels and upsamples it. Adapted from Xie et al. [31].

Note 5.1

Remark on a possible error in [31]. When implementing the SegFormer architecture using the code provided by the author on [github](#), I noticed that the position of the overlap patch merging relative to the composite block MixFFN + EfficientSelfAttention was swapped compared to what is represented in the schematics presented here, picked from the original paper: in the code, every stage starts with a single overlap patch merging, followed by its N blocks and a normalisation. The implementation in our project is done accordingly to what can be found on the authors repository.

Efficient self-attention. A feature map of size $h \times w$ is a sequence of hw tokens, and full self-attention would cost $O((hw)^2)$. SegFormer computes the keys and values on a reduced map: a complex convolution with kernel size and stride R , followed by a normalisation, shrinks the map by R in each direction before the keys and values are projected. The queries keep the full resolution, so the output has one token per position, while the cost drops to $O((hw)^2/R^2)$. The attention itself is the complex attention (2.6). Figure 5.3 details the layer.

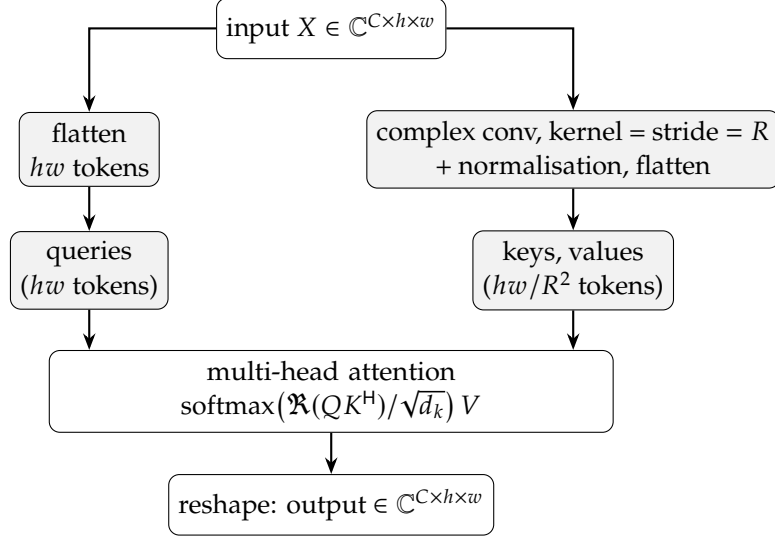


Figure 5.3: Efficient self-attention: the queries are computed at full resolution, the keys and values on a map reduced by a factor R in each direction.

Mix-FFN. The feed-forward part of each block is a point-wise complex convolution from C to C channels, a 3×3 depth-wise complex convolution that expands each channel into four with its own kernels (from C to $4C$ channels, with C groups), a CGELU, and a point-wise convolution back to C channels. The depth-wise convolution mixes each position with its neighbours and supplies the positional information that transformers otherwise obtain from positional encodings. This arrangement differs slightly from the original Mix-FFN, which expands the channels with a point-wise layer before the depth-wise convolution; the depth-wise expansion used here is lighter.

Stochastic depth. During training, the residual branch of each Mix-FFN is dropped for the whole batch with a probability that grows linearly from 0 in the first block to 0.1 in the last one [12], and rescaled by the inverse survival probability otherwise.

Configuration. Table 5.1 gives the configuration of the four stages and the sizes of their outputs for a 64×64 input patch. With these reduction ratios, every stage computes its keys and values on a 2×2 map: each query attends to a summary of the whole stage in four tokens.

Table 5.1: Configuration of the encoder and output sizes for a 64×64 input. Patch merging is given as kernel size / stride.

Stage	Patch merging	Channels	Heads	Reduction R	Output	Parameters
1	7 / 4	8	2	8	16×16	22,928
2	3 / 2	16	2	4	8×8	31,680
3	3 / 2	32	4	2	4×4	70,016
4	3 / 2	64	4	1	2×2	215,808

Two transformer blocks per stage, Mix-FFN expansion 4. Parameters are counted in real degrees of freedom.

5.3 Decoder and segmentation head

The decoder brings the four feature maps to the resolution of the first one, 16×16 . Each map is upsampled by nearest-neighbour interpolation, by a factor 8, 4, 2 and 1 respectively, and projected to 64 channels by a point-wise complex convolution; since both operations are linear and act on different axes, their order does not matter. The four maps are concatenated into 256 channels and fused by a point-wise convolution to 64 channels, followed by a modReLU and a complex batch normalisation. A last point-wise convolution produces one complex score per class, and the score map is upsampled bilinearly by a factor 4 to the 64×64 resolution of the input. Class probabilities are given by the modulus softmax (2.7).

5.4 Contrastive heads

A contrastive loss compares one vector per view, while each stage of the encoder produces a map of a different size and depth. For pre-training, every stage s receives its own projection head: a point-wise complex convolution from C_s to 128 channels, a normalisation and a modReLU, followed by a global average pooling over the spatial positions and a normalisation to unit norm (Figure 5.4). Each head turns the feature map of its stage into an embedding $z_s \in \mathbb{C}^{128}$ of the same dimension, whatever the resolution of the stage, and the averaging makes the embedding independent of where a structure lies inside the patch. The contrastive objectives of Chapter 6 combine one loss per stage.

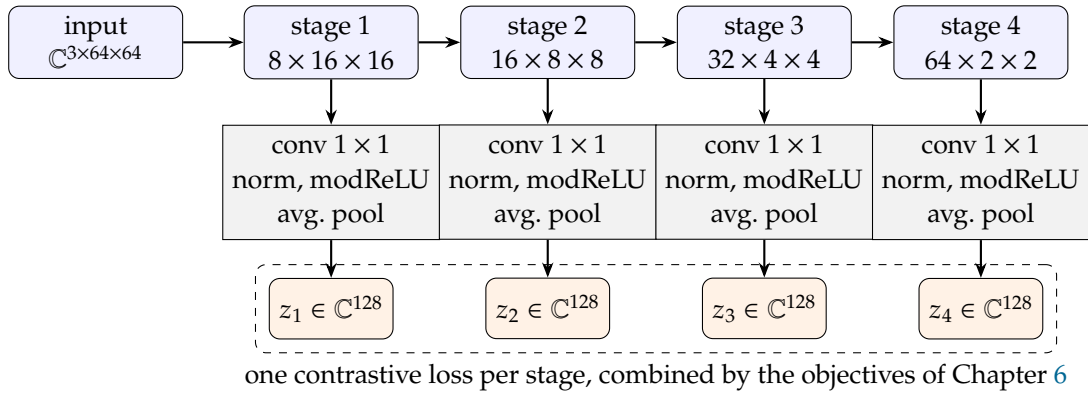


Figure 5.4: Pre-training configuration: each encoder stage feeds its own projection head, which produces a unit-norm complex embedding of dimension 128. Sizes are given as channels $\times$ height $\times$ width.

After pre-training, only the encoder is transferred to the segmentation network: its 164 parameter tensors are copied, while the decoder and the segmentation head start from a random initialisation, and the projection heads are discarded.

5.5 Size of the networks

Table 5.2 gives the number of parameters of the networks which can be either complex or real valued, a natural consequence is that a complex parameter must be counted twice.

Table 5.2: Number of parameters, in real degrees of freedom (a complex parameter counts twice).

Component	Parameters
Encoder (transferred by pre-training)	340,432
Decoder: per-stage projections	15,872
Decoder: fusion and segmentation head	34,063
Segmentation network	390,367
Contrastive heads (pre-training only)	33,796

It is a small network, which we think is appropriate for a dataset of a few thousand patches.

Chapter 6

Training objectives

Pre-training attaches one contrastive loss to each of the $S = 4$ stages of the encoder. This chapter studies how to combine hierarchical embeddings in a contrastive loss. This work led to implementing two of them, one has a simple intuitive explanation, the other that plays a similar role is grounded in literature being directly inspired by [15].

6.1 Multi-stage contrastive learning

Traditionally ViT (Vision Transformers) introduced in [6] have an encoder that produce a single embedding as the output passed to an MLP head. This embedding summarizes completely what the encoder learnt from data in a single place. Hence, a natural configuration for contrastive learning is to evaluate an InfoNCE on the two 1D embedding produced for two contrastive views. However in the case of an architecture of several stages, each producing a feature map that is not 1D, it is clear that contrastive losses must be adapted. A step in this direction was done in the previous section and more specifically in Figure 5.4 where we described how we could design contrastive head for the SegFormer architecture. From those heads, the NT-Xent loss (3.1) with the complex similarity (3.4) gives a loss ℓ_s .

One could wonder why we aim at computing separate losses for each stage of the encoder in the very first place, there are three reasons. First the decoder uses all four feature maps, so all four should be trained to carry information, and training only the last stage would leave the shallow stages to be shaped indirectly. Second, the model should not be penalized when struggling to learn the weights of a noisy stage, in imaging noise can happen because of several factors, such as the noise scale that makes some resolution scales unattainable for any set of weights. Finally combining losses with a set of weights chosen by hand is not computationally feasible (tuning four weights by hand would require one pre-training and one fine-tuning per candidate), another problem would be generality of the approach (the result would be specific to one architecture and one dataset).

We therefore let the weights be learnt, and study two ways of doing so.

6.2 Softmax weights regularised towards uniformity

The first objective holds one free logit θ_s per stage and reads the weights through a softmax, $w = \text{softmax}(\theta)$, so that $w_s \geq 0$ and $\sum_s w_s = 1$. Minimising $\sum_s w_s \ell_s$ alone would put all the weight on the stage with the smallest loss: the other stages would stop receiving gradient and stop being trained. The weights are therefore regularised towards the uniform distribution $\mathcal{U}$

over the S stages, $\mathcal{U}_s = 1/S$, through the Kullback–Leibler divergence

$$\text{KL}(w \parallel \mathcal{U}) = \sum_{s=1}^S w_s \log(S w_s), \quad (6.1)$$

which is non-negative and vanishes if and only if w is uniform. The objective, called KLU_{unif} in the following, is

$$\mathcal{L}_{\text{KLU}_{\text{unif}}} = \sum_{s=1}^S w_s \ell_s(\tau) + \lambda \text{KL}(w \parallel \mathcal{U}), \quad w = \text{softmax}(\theta), \quad (6.2)$$

with a temperature fixed at $\tau = 0.1$ for all stages and a regularisation strength $\lambda \geq 0$.

Proposition 6.1. *For fixed stage losses $\ell_1, \dots, \ell_S$ and $\lambda > 0$, the weights that minimise (6.2) over the simplex are*

$$w_s^\star = \frac{\exp(-\ell_s/\lambda)}{\sum_k \exp(-\ell_k/\lambda)}. \quad (6.3)$$

Proof. Let $Z = \sum_k \exp(-\ell_k/\lambda)$, so that $\ell_s = -\lambda \log(Z w_s^\star)$ for every s . For any w on the simplex, using $\sum_s w_s = 1$,

$$\sum_s w_s \ell_s = -\lambda \sum_s w_s \log w_s^\star - \lambda \log Z, \quad \lambda \text{KL}(w \parallel \mathcal{U}) = \lambda \sum_s w_s \log w_s + \lambda \log S,$$

and summing the two, the objective (6.2) reads

$$\lambda \text{KL}(w \parallel w^\star) + \lambda \log(S/Z).$$

The second term does not depend on w , and $\text{KL}(w \parallel w^\star) \geq 0$ with equality if and only if $w = w^\star$. Hence $w^\star$ is the unique minimiser. $\square$

The regularisation strength therefore plays the role of a temperature on the stage losses. For $\lambda \rightarrow 0$, the weights concentrate on the stage with the smallest loss; for $\lambda \rightarrow \infty$, they become uniform, and in between, λ sets how much a difference of loss between stages changes their weights. It is also clear to see what this objective rewards: the stages that are easiest for the contrastive task, those with the smallest loss, receive the largest weight. This is a heuristic choice, and not necessarily the right one, since a stage with a large loss may be precisely the one that needs training.

6.3 Stage weights as learnable temperatures

The second objective starts from the probabilistic reading of Section 3.2: the loss of a stage is the negative log-likelihood of a categorical model. Two distributions over the candidates of an anchor i are involved. The label, being either 1 or 0 for all sample, puts all its mass on the positive i^+ . The prediction of the model is the categorical distribution (3.2), which this chapter writes with the inverse temperature $\beta = 1/\tau$ as $p_\beta(k \mid i)$, and abbreviates as $q_k = p_{\beta_0}(k \mid i)$ at a fixed inverse temperature β_0 . It is a softmax of the similarities, so every candidate receives a probability $q_k > 0$. The loss is the cross-entropy between the two,

$-\sum_k \mathbf{1}[k = i^+] \log q_k = -\log q_{i^+}$, which is well defined since $q_{i^+} > 0$. KLUnif (6.2) multiplies this loss by a stage weight w_s .

We show in what is coming that a weighted loss has no probabilistic counterpart; the correct way to weight a stage is to tune its temperature.

Proposition 6.2. *Let i be an anchor, i^+ its positive and $\text{sim}_k = \text{sim}(z_i, z_k)$ its similarity (3.4) with candidate k . Let $q_k = p_{\beta_0}(k | i) \propto \exp(\beta_0 \text{sim}_k)$ be the distribution that the model predicts for i over its $K \geq 2$ candidates at inverse temperature β_0 , with $\sum_k q_k = 1$, and let $\beta > 0$ be a weight on the loss $-\log q_{i^+}$. The weighted loss $\beta(-\log q_{i^+})$ is the negative log-likelihood of a normalised distribution only if $\beta = 1$. Renormalising q^β gives the model at inverse temperature $\beta\beta_0$, $\tilde{q}_k = p_{\beta\beta_0}(k | i)$:*

$$\tilde{q}_k = \frac{q_k^\beta}{\sum_{k'} q_{k'}^\beta} = \frac{\exp(\beta\beta_0 \text{sim}_k)}{\sum_{k'} \exp(\beta\beta_0 \text{sim}_{k'})}, \quad -\log \tilde{q}_{i^+} = \beta(-\log q_{i^+}) + \log \sum_k q_k^\beta. \quad (6.4)$$

Proof. $\beta(-\log q_{i^+}) = -\log q_{i^+}^\beta$ is the negative log-likelihood of the family $(q_k^\beta)_k$. A softmax gives every candidate a probability $0 < q_k < 1$, so for $\beta > 1$, $q_k^\beta < q_k$ for every k and $\sum_k q_k^\beta < 1$, while for $\beta < 1$ the inequalities are reversed and $\sum_k q_k^\beta > 1$: the family sums to one only for $\beta = 1$. Since $q_k^\beta \propto \exp(\beta\beta_0 \text{sim}_k)$, normalising gives the first equality of (6.4), and taking the negative log-likelihood gives the second. $\square$

A weight on an InfoNCE loss is only a genuine likelihood once the omitted normalisation term $\log \sum_k q_k^\beta$ is restored, and the likelihood obtained in this way is simply InfoNCE at a rescaled temperature. Learnable stage weights on likelihood losses are therefore expressed consistently as learnable temperatures.

Our final objective. Each stage receives its own inverse temperature $\beta_s = 1/\tau_s$, learnt through $\log \beta_s$, and its loss is InfoNCE at that temperature. The objective, called LearnableTemp, is

$$\mathcal{L}_{\text{LearnableTemp}} = \sum_{s=1}^S \ell_s(1/\beta_s) + \lambda \text{KL}(\tilde{\beta} \| \mathcal{U}), \quad \tilde{\beta}_s = \frac{\beta_s}{\sum_k \beta_k}. \quad (6.5)$$

Every term of the sum is the negative log-likelihood of a categorical model, and the sum is the negative log-likelihood of one independent model per stage. The independence permitting the summation is an approximation, since all stages share the encoder, but each term is now a well-defined likelihood. The role of the regularisation follows from two properties of β_s .

What drives the temperature. Consider the likelihood alone, $\lambda = 0$, and one anchor i of a stage. Writing sim_k for the similarity of the anchor with candidate k , the loss of the anchor at inverse temperature β is

$$\ell_i(\beta) = -\log p_\beta(i^+ | i) = -\beta \text{sim}_{i^+} + \log \sum_{k \neq i} e^{\beta \text{sim}_k}.$$

Differentiating the logarithm of the sum by the chain rule gives

$$\begin{aligned}\frac{\partial \ell_i}{\partial \beta} &= -\text{sim}_{i^+} + \frac{\sum_{k \neq i} \text{sim}_k e^{\beta \text{sim}_k}}{\sum_{k' \neq i} e^{\beta \text{sim}_{k'}}} = -\text{sim}_{i^+} + \sum_{k \neq i} p_\beta(k | i) \text{sim}_k \\ &= -\text{sim}_{i^+} + \mathbb{E}_{k \sim p_\beta}[\text{sim}_k],\end{aligned}\tag{6.6}$$

since $e^{\beta \text{sim}_k} / \sum_{k' \neq i} e^{\beta \text{sim}_{k'}}$ is exactly $p_\beta(k | i)$: the derivative is the average of the similarities weighted by the probabilities of the model. The expectation runs over all candidates, the positive included. A gradient step changes β by $\eta (\text{sim}_{i^+} - \mathbb{E}_{k \sim p_\beta}[\text{sim}_k])$: the inverse temperature increases, and the temperature decreases, when the positive is more similar to the anchor than the average candidate under the model, this accounts for the model becoming more confident in its predictions.

Note 6.1

Let's think of what conceptually differs between our two losses we previously defined. In the first KLUif the weights multiply the gradient of the stage while in LearnableTemp it scales the gradient and focuses the loss on the negatives that are difficult to identify at this stage. This second loss has both a stronger mathematical background since it is a genuine likelihood but also adapts the difficulty of each stage during training. It can be seen in Proposition 6.1 where the optimal weights only depend on the loss magnitude at a given stage while in (6.6) the temperature evolves following the quality of the separation between negatives and positives learned by the stage, a collapsed stage (where all similarities are equal to their mean) stops updating its temperature but the encoder still gets gradients anyway. Conceptually and in practice the second approach seems more promising.

Implementation. The inverse temperatures are initialised at $\beta_s = 10$, that is $\tau_s = 0.1$, the fixed temperature of KLUif, so that the two objectives start from the same per-stage losses. They are optimised with the network, with the same optimiser, and $\log \beta_s$ is clipped to $[-5, 5]$ after every step, which keeps τ_s between 0.007 and 148. The sum in (6.5) is S times larger than the weighted average in (6.2) at initialisation; with the per-parameter normalisation of Adam, such a constant factor has essentially no effect on the updates of the network, although it makes the regularisation term S times weaker relatively to the likelihood than in (6.2).

6.4 Supervised objective

The segmentation network is trained with the focal loss of Lin et al. [19], which addresses the class imbalance of Table 1.1. For a labelled pixel of class y whose predicted probability (2.7) for that class is p_y ,

$$\mathcal{L}_{\text{focal}} = -\alpha_y (1 - p_y)^\gamma \log p_y,\tag{6.7}$$

and the loss of a batch is the average over its labelled pixels; unlabelled pixels are ignored, and a batch without any labelled pixel contributes zero. The factor $(1 - p_y)^\gamma$ reduces the contribution of pixels that are already well classified, so that training focuses on hard pixels (Figure 6.1); we use $\gamma = 1.2$. The class weights

$$\alpha = (40.0, 3.03, 27.77, 10.27, 15.24, 55.0)\tag{6.8}$$

for mountain, water, vegetation, high-density urban, low-density urban and developed areas are approximately proportional to the inverse class frequencies: relative to water, they are 13.2,

9.2, 3.4, 5.0 and 18.2, against inverse frequency ratios of 13.1, 10.1, 3.5, 4.9 and 18.6 computed from Table 1.1.

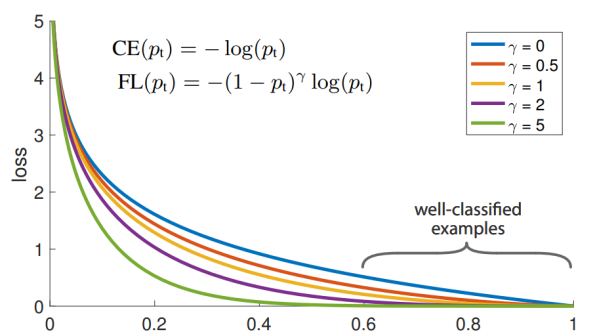


Figure 6.1: Focal loss as a function of the probability of the true class, for several values of γ ; $\gamma = 0$ gives the cross-entropy. Adapted from Lin et al. [19].

Chapter 7

Experimental protocol

The experiments ask whether contrastive pre-training on the unlabelled part of the scene helps the segmentation network, in final performance, in speed of convergence and when labels are scarce, and which of the two objectives of Chapter 6 helps most. The encoder is pre-trained with KLU_{unif} and LearnableTemp for several regularisation strengths λ , then transferred to the segmentation task, frozen or fine-tuned, on growing shares of the labelled patches. Each pre-trained encoder is compared with a randomly initialised one, frozen or trained in the same way. Section 7.5 details which configurations were repeated.

7.1 Hardware

The code is written in Python 3.13 with PyTorch 2.12 and torchcvnn 0.9.4 (installed in april 2025). All experiments run on a single laptop GPU, an NVIDIA GeForce RTX 4060 Laptop 8 GB which explains why we ruled out heavily parametrized architectures or long trainings. The data was read from an external SSD (personal Samsung T7).

A pre-training epoch over the 40,958 patches takes about seven minutes, and a pre-training run of twelve epochs about an hour and a half; a supervised run takes a few minutes only.

7.2 Pre-training

The encoder is pre-trained on the 40,958 patches of the unlabelled regions (Section 4.2), shuffled at every epoch, with the augmentations of Section 4.4. Table 7.1 gives the hyperparameters. The learnable parameters of the objective, the logits θ_s of KLU_{unif} or the inverse temperatures $\log \beta_s$ of LearnableTemp, are optimised together with the network. No data is held out during pre-training: the contrastive loss is not a reliable criterion to select a checkpoint for a downstream task, so the pre-training is a fixed budget and the encoder of the last epoch is transferred.

Table 7.1: Pre-training hyperparameters.

Patches, size	40,958, 64×64 , no overlap
Batch	$N = 16$ patches, $2N = 32$ views, 30 negatives per anchor
Epochs	12 (2,560 steps per epoch)
Optimiser	AdamW [23], $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$
Learning rate	10^{-4} , cosine annealing to 0 over the 12 epochs [22]
Weight decay	5×10^{-4}
Gradient clipping	global norm 2.5 (network parameters)
Stochastic depth	linear from 0 to 0.1 over the 8 blocks
Embedding dimension	128 (complex) per stage
Temperature	KLUnif: $\tau = 0.1$ fixed; LearnableTemp: τ_s initialised at 0.1
Checkpoint	last epoch

7.3 Supervised training and fine-tuning

The segmentation network is trained on the 2,038 training patches of the lattice split (Section 4.3) with the focal loss of Section 6.4, either from a random initialisation (the baseline) or from a pre-trained encoder (fine-tuning), in which case the decoder and the segmentation head are randomly initialised and all the parameters are trained. Both settings use exactly the same data, hyperparameters (Table 7.2) and number of steps, and no augmentation. After every epoch, the network is evaluated on the validation patches, and the checkpoint with the highest validation mIoU is kept. The validation loss is not used for this purpose: with class weights up to 55 (Section 6.4), it is dominated by the few pixels of the rarest classes that are misclassified with confidence, and it rose during training while the mIoU kept improving, so that selecting on it kept checkpoints of the first epochs. The test patches are used once, at the end, on the selected checkpoint.

The learning rate was chosen on the validation set among 5×10^{-3} , 10^{-3} and 3×10^{-4} , for the baseline and for fine-tuning separately, with two seeds each. The largest value gave the highest mean validation mIoU in both cases, 42.9 against 37.1 and 26.0 for the baseline, 55.0 against 52.7 and 48.9 for fine-tuning, and is used throughout.

Table 7.2: Hyperparameters of the supervised training, identical for the baseline and for fine-tuning.

Patches	2,038 train, 679 validation, 680 test
Batch	16 patches
Epochs	24 (128 steps per epoch)
Optimiser	AdamW, learning rate 5×10^{-3} , weight decay 5×10^{-4}
Schedule	cosine annealing to 0 over the 24 epochs
Gradient clipping	global norm 2.5
Loss	focal loss, $\gamma = 1.2$, class weights of Section 6.4
Model selection	highest validation mIoU

7.4 Metrics

All metrics are computed from the confusion matrix $M \in \mathbb{N}^{6 \times 6}$ accumulated over all the labelled pixels of the test patches, where M_{ab} counts the pixels of true class a predicted as

class b ; unlabelled pixels are excluded. We list below the evaluation metrics used in this research:

- With $n = \sum_{a,b} M_{ab}$, the overall accuracy is $OA = \sum_a M_{aa}/n$.
- For each class c ,
 - the precision $P_c = M_{cc}/\sum_a M_{ac}$
 - the recall $R_c = M_{cc}/\sum_b M_{cb}$
 - the F1 score $F_c = 2P_cR_c/(P_c + R_c)$

- The intersection over union is computed as:

$$IoU_c = \frac{M_{cc}}{\sum_a M_{ac} + \sum_b M_{cb} - M_{cc}} \quad (7.1)$$

- The macro F1 score and the mean IoU (mIoU) are their averages over the six classes, which gives rare classes the same importance as water that clearly dominates in the dataset.

Because of the class imbalance, the mIoU and the macro F1 score are the primary metrics; the overall accuracy is dominated by water.

7.5 Experiments

All experiments are seeded (Python, NumPy and PyTorch generators), PyTorch’s deterministic algorithms were also enforced for maximal reproducibility (please find information at this link: <https://pytorch.org/docs/stable/notes/randomness.html>).

The seed sets the initialisation, the order of the batches and the augmentations. Each configuration is run with one to three seeds, as detailed below, and the results are given as the mean and the sample standard deviation over seeds.

Experiment 1: weighting of the stages. The encoder is pre-trained with each of the two objectives of Chapter 6, KLUUnif and LearnableTemp, for several regularisation strengths λ . The value $\lambda = 3$ is run with two seeds for KLUUnif and three for LearnableTemp; the other values with one seed. The quantities of interest are the evolution of the stage weights w_s and $\tilde{\beta}_s$, of the temperatures τ_s and of the loss during pre-training.

The grid is $\lambda \in \{0, 0.03, 0.3, 3\}$. It spreads over several orders of magnitude because the stage losses differ by about 10^{-2} at the end of pre-training (Proposition 6.1): values such as $\lambda = 1$ and $\lambda = 3$ would both give nearly uniform KLUUnif weights at that point.

Experiment 2: transfer to segmentation. Every encoder of Experiment 1 is fine-tuned on the segmentation task, with the seed of its pre-training. The three encoders of Experiment 3 are also fine-tuned with seeds 1 and 2, from their pre-training with seed 0. The baseline is trained from scratch with seeds 0, 1 and 2. The comparison between the fine-tuned networks

and the baseline measures the effect of pre-training; the comparison between objectives and values of λ measures the effect of the weighting.

Experiment 3: label efficiency. The supervised training is repeated on 10%, 25% and 50% of the training patches. The patches are drawn from the seed of the run: the networks of a given seed train on the same patches, and the subsets of a seed are nested. The validation and test patches are unchanged. The number of gradient steps is that of a run on every patch: with a share f of the patches, the network is trained for $24/f$ epochs and validated every $1/f$ epochs, so that every share selects among the same number of checkpoints. Four modes are compared, with seeds 0, 1 and 2:

- the baseline, trained from scratch;
- a frozen random encoder, of which only the decoder and the segmentation head are trained, the control that tells what they learn alone;
- a frozen pre-trained encoder, which scores the pre-trained representation itself, whatever a fine-tuning would make of it;
- a fine-tuned pre-trained encoder.

Note 7.1

Important remarks.

- The batch normalisations of a frozen encoder keep estimating their running statistics during supervised training, since the pre-training checkpoints contain none. See Section 9.2: during pre-training, all normalisation layers used batch statistics and kept no running statistics.
- The encoder of each objective was selected on the validation set: every encoder of Experiment 1 with seed 0 was frozen and trained on 10% of the patches, and the highest validation mIoU designated KLU_{unif} with $\lambda = 0$ and LearnableTemp with $\lambda = 0.3$. The eight encoders differed by about two points only, less than the spread between seeds; LearnableTemp with $\lambda = 0$ was therefore added, to compare both objectives at the same λ . $\lambda = 0$ is also an interesting case where the weights/temperatures learned by both objectives are evolving freely.
- The three seeds of a pre-trained mode share one pre-trained encoder to measure only the variability of the fine-tuning.

Chapter 8

Results

8.1 Pre-training

Figure 8.1 shows the pre-training loss. Both objectives decrease quickly during the first two epochs and then slowly, and the decrease has not completely stopped after twelve epochs. The magnitudes of the two objectives must not be compared with each other: as we said in Chapter 6, KLUrif is a weighted average of four losses at a fixed temperature, LearnableTemp a sum of four losses at learnt temperatures and a valid likelihood.

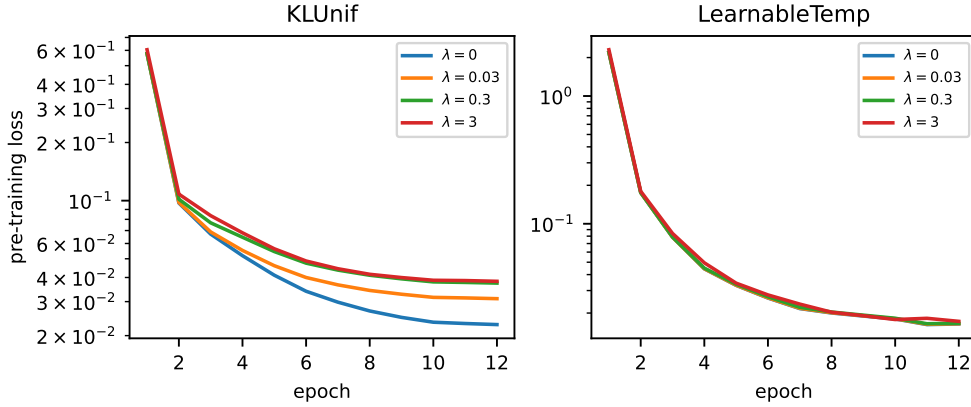


Figure 8.1: Pre-training loss per epoch, averaged over seeds, for each objective and value of λ (logarithmic scale).

However it is interesting to note that the loss evolution in the second case does not vary much with the selected penalisation, we need to take a closer look to the stage weights learned during pre-training which is what Figure 8.2 is about. It shows the evolution of the stage weights: the softmax weights w_s for KLUrif and the relative inverse temperatures $\tilde{\beta}_s$ for LearnableTemp, which play the same role (Section 6.3).

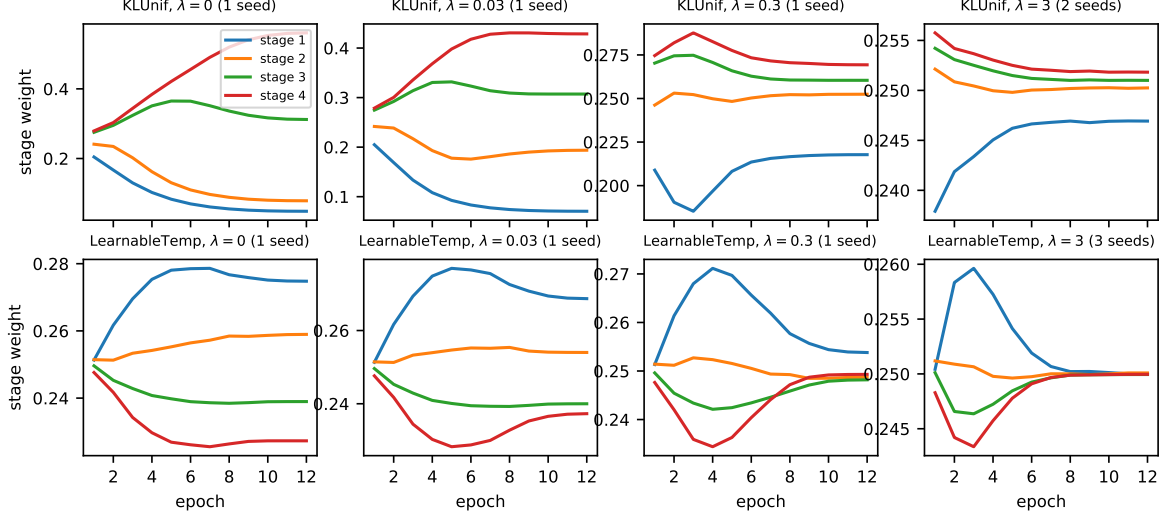


Figure 8.2: Stage weights during pre-training, averaged over seeds: w_s for KLUUnif (top), $\tilde{\beta}_s$ for LearnableTemp (bottom), for each value of λ .

General remark. As one could have feared, whilst the inverse temperature and the weights of KLUUnif play a similar role, both have a very different dynamic for each level of penalisation.

Effect of regularisation and differences of behavior between objectives. At $\lambda = 3$ with KLUUnif, the weights start between 0.238 and 0.256 after the first epoch and settle closely to 0.25 between 0.247 and 0.252 (uniformly distributed weights as we could have expected from a strong regularization): the first stage receives the smallest weight and the deeper stages slightly more, identically on both seeds. By Proposition 6.1, this ordering means that the first stage has the largest contrastive loss, which is plausible for the stage with the lowest-level features and the fewest channels. With LearnableTemp, the relative inverse temperatures start from uniformity during the first epochs, the first stage reaching 0.26 at the third epoch while the last one falls to 0.243, and return within 10^{-4} of the uniform distribution after the eighth epoch. At $\lambda = 3$, the regularisation thus dominates both objectives at the end of pre-training. We need to observe less regularised behaviors to find more patterns. However, even with strong regularisation we can see that the weights / inverse temperatures have a phase where they evolve monotonically, this suggests that at the beginning of the training, the objective decreases better by tuning the weights that since some stages seem to be learning more than others but after some epochs, their respective loss is not decreasing fast enough for compensating the strength of the regularisation. This also means that without regularisation we should observe the weights stabilizing after some epochs. When looking at $\lambda = 0$ this hypothesis seems to confirm. Another behavior that deserves to be noted is that the two objectives order their weights in a reversed order, this is what was explained in Note 6.1, this is also visible on the figure below that shows the evolution of the temperatures during pre-training.

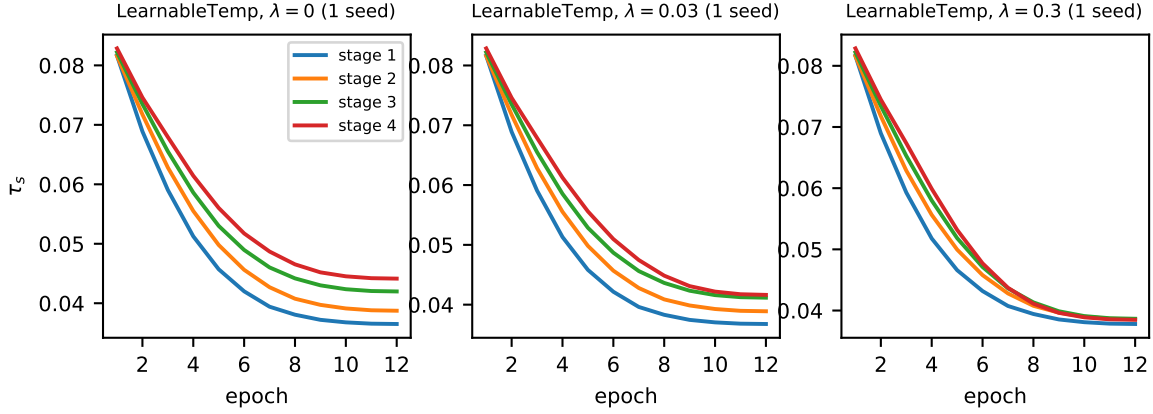


Figure 8.3: Per-stage temperatures τ_s of LearnableTemp during pre-training, for each value of λ .

8.2 Transfer to segmentation

Table 8.1 reports the test metrics of the baseline and of the fine-tuned networks, and Table 8.2 the IoU of each class. These figures were produced for fine-tunings performed on the whole annotated dataset which represents around 2 000 of 64×64 patches with about two fifths of their pixels being unlabelled.

Table 8.1: Test results on the PolSF labelled area (in %), mean $\pm$ standard deviation over seeds.

Experiment	Seeds	OA	mIoU	macro F1
From scratch (baseline)	3	81.4 ± 8.4	46.6 ± 12.6	57.7 ± 13.3
Fine-tuned, KLUUnif, $\lambda = 0$	3	85.6 ± 0.7	51.7 ± 1.9	62.7 ± 1.6
Fine-tuned, KLUUnif, $\lambda = 0.03$	1	85.9	51.6	63.4
Fine-tuned, KLUUnif, $\lambda = 0.3$	1	87.1	54.9	65.9
Fine-tuned, KLUUnif, $\lambda = 3$	2	85.0 ± 4.2	51.5 ± 3.9	63.1 ± 1.6
Fine-tuned, LearnableTemp, $\lambda = 0$	3	86.9 ± 1.3	54.1 ± 4.6	66.1 ± 5.1
Fine-tuned, LearnableTemp, $\lambda = 0.03$	1	88.0	54.5	64.6
Fine-tuned, LearnableTemp, $\lambda = 0.3$	3	85.6 ± 1.9	49.5 ± 3.9	60.1 ± 3.5
Fine-tuned, LearnableTemp, $\lambda = 3$	3	86.7 ± 1.5	55.5 ± 2.6	68.1 ± 2.2

Table 8.2: Test IoU per class (in %), mean over seeds.

Experiment	Mountain	Water	Veget.	HD urban	LD urban	Developed
From scratch (baseline)	43.4	96.8	31.8	47.3	54.4	5.9
Fine-tuned, KLUUnif, $\lambda = 0$	49.7	98.2	33.4	59.1	66.1	3.8
Fine-tuned, KLUUnif, $\lambda = 0.03$	39.1	99.3	38.7	56.5	67.6	8.6
Fine-tuned, KLUUnif, $\lambda = 0.3$	46.5	96.6	41.0	66.0	73.0	6.1
Fine-tuned, KLUUnif, $\lambda = 3$	51.5	97.7	35.8	54.1	62.2	7.7
Fine-tuned, LearnableTemp, $\lambda = 0$	47.6	99.0	37.7	61.0	64.7	14.6
Fine-tuned, LearnableTemp, $\lambda = 0.03$	54.2	98.4	41.3	63.0	70.3	0.0
Fine-tuned, LearnableTemp, $\lambda = 0.3$	46.3	98.7	32.3	56.6	63.2	0.1
Fine-tuned, LearnableTemp, $\lambda = 3$	44.9	98.0	43.1	61.7	66.5	19.1

Figure 8.4 shows the validation mIoU of the baseline and of two of the fine-tuned encoders during training, for each seed.

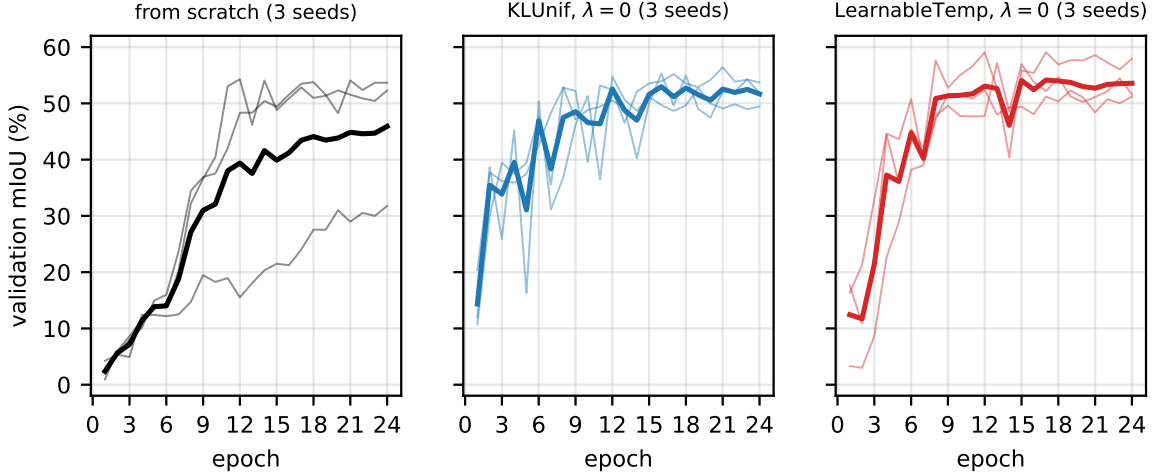


Figure 8.4: Validation mIoU per epoch with every training patch, for the baseline and the encoders of Section 8.3: one thin line per seed, their mean in bold.

On the efficiency of pre-training. From the results shown above it makes no doubt that pre-training does not harm the performance of the model when trained on the labeled data. They neatly reduce the variance in training but also the convergence of the model: it achieves the same validation mIoU as the baseline in about 10 epochs in only 2 to 4. This suggests the contrastive learning was effectively building inner representations that served during training. However the performance reaches a ceiling after 24 epochs, probably due mostly to the size of the model which is quite modest compared to other classical semantic segmentation architectures. These results however are a good sign for the application of pre-training in frugal learning: learning with a few annotated data. Since our model is small in size, we run experiments with fractions of the dataset to evaluate whether a pre-trained model performs better when fine-tuned with very little data; which is a relevant application for SAR imaging.

On the differences between objective functions. None seems to perform significantly better, except that runs made with KLUnif converges even faster than the LearnableTemp in average. This could suggest that hierarchical pre-training is better done when we allow the hierarchical objective to somewhat "forget" a noisy stage.

Note 8.1

Unseen but important. Another studied set of values was the deviation of the pre-trained weights after fine-tuning. We noted during our experiments that weights in the encoder were significantly changed in average after 24 epochs of fine-tuning by about $\approx 80\%$. The validation metrics speak for themselves, showing that pre-training is indeed useful, however this phenomenon needs to be taken into account while being difficult to interpret. It would require a whole set of experiments for its own sake.

8.3 Label efficiency

Figure 8.5 and Table 8.3 report the test mIoU of each training mode when only a share of the training patches is used. This share goes from 10% to 100% of the total dataset.

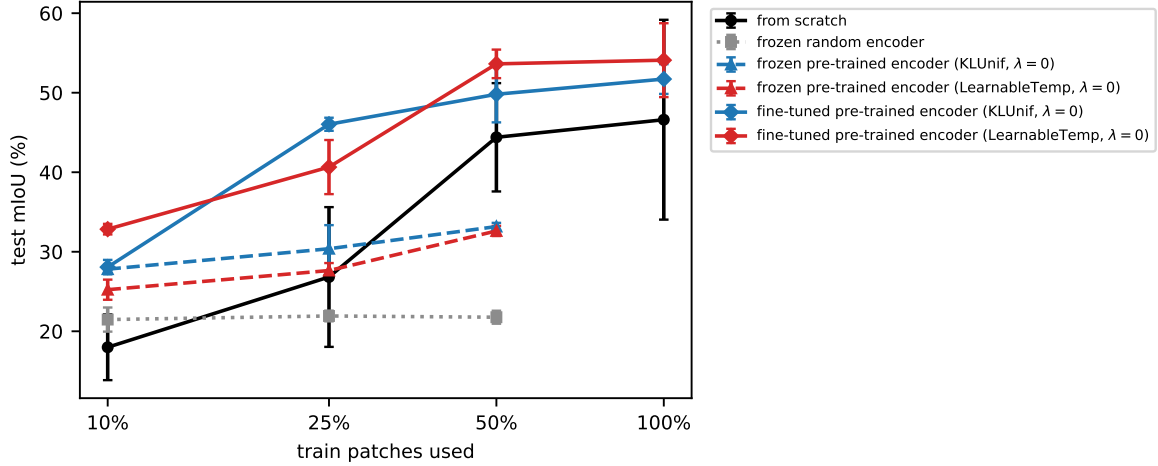


Figure 8.5: Test mIoU against the share of the training patches used, mean $\pm$ standard deviation over three seeds: training from scratch, a frozen random encoder, and the encoders pre-trained with KLUnif ($\lambda = 0$) and LearnableTemp ($\lambda = 0$), frozen (dashed) or fine-tuned (solid). Each pre-trained encoder comes from a single pre-training run: the spread measures the variability of the fine-tuning (initialisation of the decoder, order of the batches, subset of patches), not that of the pre-training.

A success for pre-trained models in frugal cases. The pre-training also has a variance reduction effect on the test predictions, whatever the fraction on which it was fine-tuned, except for LearnableTemp with $\lambda = 0.3$ at 10% and 25%. It also improves the absolute results obtained on all fractions of the dataset in the complete fine-tuning setting and performs significantly better than a randomly initialized frozen encoder when frozen itself. In the 10% fraction case, fine-tuning a pre-trained encoder shows a 56% to 82% increase in test mIoU over a randomly initialized baseline. An interesting comparison we can draw is between the fully trained baseline (encoder and decoder are both initialized randomly then trained in a supervised manner on 10% of the data) and the frozen encoders on the same share: a trained decoder on top of a frozen pre-trained encoder performs 40% to 54% better than the fully trained baseline when fine-tuned on only 10% of an already small dataset. Our comparisons do not prove any major difference in the quality of the learned representations during contrastive pre-training, while it seems that LearnableTemp has an edge over KLUnif on the 10% and 50% shares, where the difference between the two is beyond standard deviation, whereas KLUnif is ahead on the 25% share and the two are within standard deviation on the 100% share.

Table 8.3: Test mIoU (in %) per training mode and share of the training patches, mean $\pm$ standard deviation over seeds, number of seeds in parentheses.

Mode	10%	25%	50%	100%
From scratch	18.0 $\pm$ 4.1 (3)	26.8 $\pm$ 8.8 (3)	44.4 $\pm$ 6.8 (3)	46.6 $\pm$ 12.6 (3)
Frozen random encoder	21.5 $\pm$ 1.5 (3)	21.9 $\pm$ 0.7 (3)	21.8 $\pm$ 0.8 (3)	–
Frozen pre-trained encoder (KLUnif, $\lambda = 0$)	27.8 $\pm$ 0.6 (3)	30.4 $\pm$ 3.0 (3)	33.2 $\pm$ 0.5 (3)	–
Frozen pre-trained encoder (LearnableTemp, $\lambda = 0$)	25.2 $\pm$ 1.3 (3)	27.6 $\pm$ 0.9 (3)	32.6 $\pm$ 0.6 (3)	–
Frozen pre-trained encoder (LearnableTemp, $\lambda = 0.3$)	25.4 $\pm$ 1.6 (3)	28.4 $\pm$ 0.4 (3)	32.9 $\pm$ 0.8 (3)	–
Fine-tuned pre-trained encoder (KLUnif, $\lambda = 0$)	28.1 $\pm$ 0.9 (3)	46.0 $\pm$ 0.8 (3)	49.8 $\pm$ 3.5 (3)	51.7 $\pm$ 1.9 (3)
Fine-tuned pre-trained encoder (LearnableTemp, $\lambda = 0$)	32.8 $\pm$ 0.7 (3)	40.7 $\pm$ 3.4 (3)	53.6 $\pm$ 1.8 (3)	54.1 $\pm$ 4.6 (3)
Fine-tuned pre-trained encoder (LearnableTemp, $\lambda = 0.3$)	29.1 $\pm$ 5.0 (3)	36.9 $\pm$ 13.3 (3)	52.0 $\pm$ 3.6 (3)	49.5 $\pm$ 3.9 (3)

Encoder: KLUUnif, $\lambda = 0$; predictions shown on the labelled pixels only

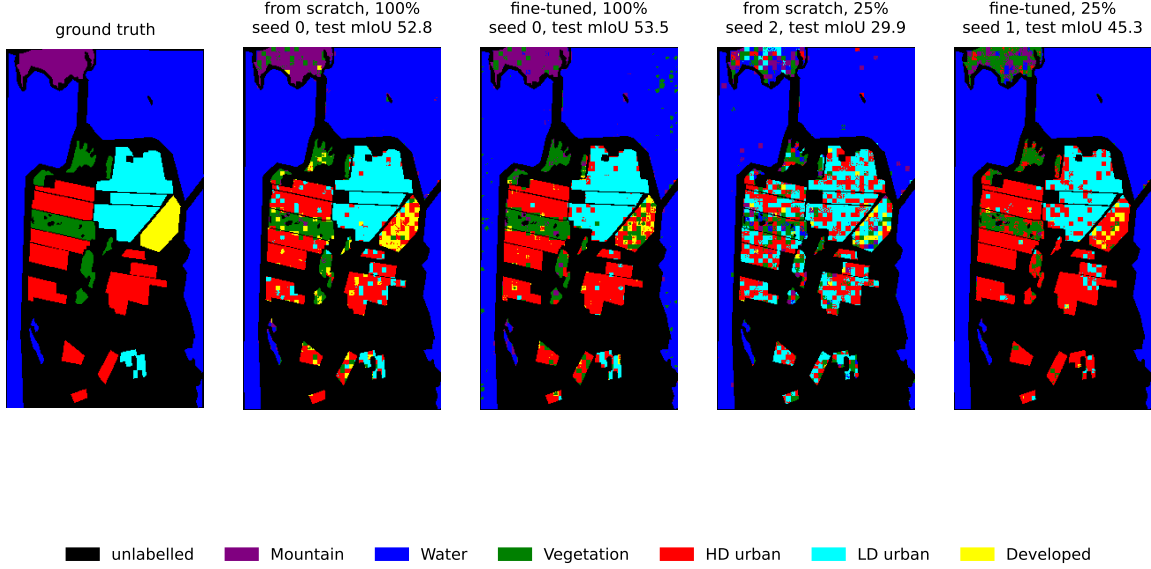


Figure 8.6: Ground truth and predictions over the labelled area, on the labelled pixels, trained from scratch and fine-tuned, with every training patch and with 25% of them. Each prediction comes from the seed whose validation mIoU is the median of the three. The area includes the training patches: the figure illustrates the typical errors, the scores are those of the test split.

Typical errors. In the frugal setting, patches are selected randomly from the mosaic split presented in Figure 4.1. Contrarily to our deterministic split pattern on the complete dataset, some classes might be rarely represented in the data picked for training. Rare classes are even more difficult to separate in this setting. Others are always difficult to separate such as the Developed and the High Density Urban classes.

8.4 Relation to published results

Complex-valued U-Nets have been applied to the same PolSF scene by Barrachina et al. [2], who compared real- and complex-valued networks and several polarimetric representations, and by the authors of torchcvnn [8], whose complex U-Net of 52 million parameters, trained with a focal loss, gives segmentations close to the state of the art. Their training and evaluation protocols differ from ours, in the size of the patches, the way the labelled area is split and the metrics reported, and their scores cannot be compared directly with those of Table 8.1. Our network is more than a hundred times smaller than the torchcvnn U-Net; the question studied here is not whether it outperforms these networks, but whether self-supervised pre-training on the unlabelled part of the scene improves it, all other things being equal.

Chapter 9

Discussion

9.1 Interpretation of the results

Weighting hierarchical contrastive losses. The two objectives order the stages in opposite ways because they deal differently with the same fact: the first stage is the hardest for the contrastive learning task. KLU_{unif}, whose weights follow the level of the losses (Proposition 6.1), lowers its weight; LearnableTemp has instead temperatures that follow the quality of separation between positives and negatives (6.6), sharpens it by reducing the temperature (the model becomes more confident). KLU_{unif} can also lower its objective by moving the weight onto the easiest stages without any stage improving, which is partly why its loss depends on λ while that of LearnableTemp does not. Yet these opposite weightings transfer equally well. This somewhat strange phenomenon is likely due to the hierarchy of the encoder. The loss of a deep stage is always back-propagated through all the stages before it, so that a stage weighted down keeps being trained anyway, for the needs of the deeper ones. It seems to us that at this scale the argument for LearnableTemp is its consistency as a likelihood (Proposition 6.2), not a measured gain, possibly because its independence hypothesis is violated, every stage depending on the previous ones.

Usefulness of pre-training on the unlabelled part of the scene. Pre-training mainly makes the supervised training reliable (less variance) and saves labels efficiently: on this scene, it replaces between half and three quarters of them (Section 8.3). However the gap between frozen and fine-tuned encoders shows that it acts as an initialisation that the fine-tuning evolves significantly, more than as a ready-to-use representation for a decoder, this would explain the large drift we mentioned in Note 8.1.

9.2 Limitations

Most of the limitations are related with our compute available for those experiments.

Hardware limitations. Each configuration is run with one to three seeds, because a pre-training run takes about an hour and a half on the available GPU. Differences smaller than the spread between seeds cannot be interpreted, and the standard deviations estimated from two or three runs are themselves uncertain. In addition, contrastive pre-training could not be run for too many epochs or benefit from larger batches which is typical in contrastive learning framework, see [4, 10]. We also could not sweep a wide range of parameters values for our contrastive augmentations presented in Section 4.4, which is in itself a major limitation of this work.

One scene, one sensor and spatial correlation. All the data comes from a single ALOS-2 acquisition, and the network is pre-trained and evaluated on the same scene. The pre-training never sees the labelled area, but it sees the same sensor, the same date and the same landscape. The results say whether pre-training on the rest of a scene helps to segment its labelled part, which is a realistic scenario when a small area of a large acquisition is annotated, but not whether the representations transfer to other scenes or sensors which is also a core question. Another aspect that must be taken into account when dealing with generalisation is our deterministic mosaic split presented exhaustively in Section 4.3. while it makes the evaluation independent of a random draw and identical for every model, it does not remove biases since every test patch is bordered by training patches, and neighbouring patches are strongly correlated. The absolute scores are therefore optimistic estimates of the accuracy on a new area; only the differences between models, measured on the same split, are meaningful. A stricter protocol would hold out whole blocks of the labelled area or even pick data from another dataset sharing similar labels, a trade-off discussed for spatial data by Roberts et al. [26].

Chapter 10

Conclusion

This work studied the self-supervised pre-training of a complex-valued segmentation network for polarimetric SAR, on a scene of which a very few pixels were labeled. We built a complex-valued SegFormer in which every operation is performed in $\mathbb{C}$, pre-trained its encoder by contrastive learning on the unlabelled portion of the scene, and fine-tuned it on the labelled part. Three elements were adapted to the nature of the data: primarily the augmentations producing contrastive views while preserving the physical identity in positive pairs; a contrastive similarity, the modulus of the normalised Hermitian product, ignoring the absolute phase of the embeddings; and an evaluation protocol based on a deterministic spatial split that gives the three subsets the same class balance and every model the same test data.

Our objective was to find an efficient and theoretically grounded way to combine losses produced at different stages of a hierarchical model into one single contrastive loss. We have introduced two losses KLU_{unif} and LearnableTemp that show very different behavior while confirming intuitions about each stage of our SegFormer. None of them proved a neat superiority onto the other but both succeeded in teaching the model a sound initialization that allowed both better convergence and overall performance of the model when fine-tuned entirely on all fractions of the total labeled training set.

Appendix A

Definition of the augmentations

This appendix defines precisely the augmentations of Section 4.4, as implemented. A patch is a complex tensor $\mathbf{Z} = (z_c)_{1 \leq c \leq C} \in \mathbb{C}^{C \times H \times W}$, with $C = 3$ and $H = W = 64$; every pixel is written $z_c(x, y) = \rho_c(x, y) e^{j\phi_c(x, y)}$, with modulus $\rho_c = |z_c|$ and phase $\phi_c = \arg z_c$. $\mathcal{U}(a, b)$ denotes the continuous uniform distribution and $\mathcal{U}\{a, \dots, b\}$ the discrete one. All the augmentations act on calibrated values, before the logarithmic normalisation (4.1).

A.1 Random hole

Definition A.1 (Random hole). Let $0 < a_{\min} < a_{\max} < 1$. Draw $m \sim \mathcal{U}\{1, 2\}$ holes. For each hole i , draw independently $u_i, v_i \sim \mathcal{U}(a_{\min}, a_{\max})$, set $h_i = \lfloor u_i H \rfloor$ and $w_i = \lfloor v_i W \rfloor$, and draw the corner $x_i \sim \mathcal{U}\{0, \dots, H - h_i\}$, $y_i \sim \mathcal{U}\{0, \dots, W - w_i\}$. With $R_i = \{x_i, \dots, x_i + h_i - 1\} \times \{y_i, \dots, y_i + w_i - 1\}$ and the mask $M = 1 - \mathbf{1}_{\cup_i R_i}$,

$$T_{\text{hole}}(\mathbf{Z})_c(x, y) = M(x, y) z_c(x, y). \quad (\text{A.1})$$

The pipeline uses $(a_{\min}, a_{\max}) = (0.05, 0.15)$.

The same mask is applied to every channel, so the polarimetric vector of a pixel is either kept whole or set to zero.

A.2 Intensity

Definition A.2 (Intensity). Let $\epsilon > 0$. Draw independently, for each channel, $\alpha_c \sim \mathcal{U}(1 - \epsilon, 1 + \epsilon)$, and set

$$T_{\text{int}}(\mathbf{Z})_c = \alpha_c z_c. \quad (\text{A.2})$$

The pipeline uses $\epsilon = 0.05$.

The moduli are scaled and the phases, including the phase differences between channels, are unchanged.

A.3 Speckle

Definition A.3 (Speckle). Let $b > 0$ and $k = 1/b$. Draw independently, for every channel and pixel, $n_c(x, y) \sim \Gamma(k, 1/k)$, a Gamma variable of shape k and scale $1/k$, so that $\mathbb{E}[n] = 1$ and $\text{Var}[n] = b$, and set

$$T_{\text{speckle}}(\mathbf{Z})_c(x, y) = n_c(x, y) \rho_c(x, y) e^{j\phi_c(x, y)} = n_c(x, y) z_c(x, y). \quad (\text{A.3})$$

The pipeline uses $b = 0.2$, that is $k = 5$.

Since n is real and positive, the phases are preserved. The multiplicative factor is drawn independently for each channel, so the ratios of amplitudes between channels fluctuate, as they do between independent looks.

A.4 Elastic deformation

Definition A.4 (Elastic deformation). Let $\alpha > 0$ be an amplitude and $\sigma > 0$ a smoothness. Draw two fields $d_x, d_y \in \mathbb{R}^{H \times W}$ with independent standard normal entries. Smooth each of them with an average filter of odd size κ (the smallest odd integer greater than or equal to $\lfloor 4\sigma \rfloor$) and stride 1, with zero padding, which gives $\tilde{d}_x, \tilde{d}_y$, and set

$$\Delta_x = \alpha \frac{\tilde{d}_x}{\max |\tilde{d}_x|}, \quad \Delta_y = \alpha \frac{\tilde{d}_y}{\max |\tilde{d}_y|}. \quad (\text{A.4})$$

The deformed patch samples the original one at the displaced positions,

$$T_{\text{el}}(\mathbf{Z})_c(x, y) = \mathcal{I}(\Re z_c)(x + \Delta_y, y + \Delta_x) + j \mathcal{I}(\Im z_c)(x + \Delta_y, y + \Delta_x), \quad (\text{A.5})$$

where $\mathcal{I}$ is bilinear interpolation, with zeros outside the patch. The pipeline uses $\alpha = 5$ pixels and $\sigma = 3$ ($\kappa = 13$), and applies the deformation with probability $\frac{1}{4}$.

The displacement field is the same for every channel and for the real and imaginary parts, so the polarimetric vectors are moved together; bilinear interpolation mixes neighbouring complex values, which slightly smooths the speckle.

A.5 Fourier masking

Definition A.5 (Fourier masking). For each channel independently, compute the centred spectrum $\hat{z}_c = \text{fftshift}(\mathcal{F}_2(z_c))$, draw a binary mask $M_c \in \{0, 1\}^{H \times W}$ as described below, force it to 1 on the central ellipse

$$\left(\frac{u - H/2}{r_H}\right)^2 + \left(\frac{v - W/2}{r_W}\right)^2 \leq 1, \quad r_H = \lfloor 0.1H \rfloor, \quad r_W = \lfloor 0.1W \rfloor, \quad (\text{A.6})$$

which preserves the lowest frequencies, and set

$$T_{\text{F}}(\mathbf{Z})_c = \mathcal{F}_2^{-1}(\text{ifftshift}(M_c \odot \hat{z}_c)). \quad (\text{A.7})$$

The mask is of one of three types, drawn uniformly:

- lines: $n \sim \mathcal{U}\{3, \dots, 8\}$ lines, each horizontal or vertical with probability $\frac{1}{2}$, of half-width $\omega \sim \mathcal{U}\{1, 2\}$ around a position drawn uniformly, are set to zero;
- rings: $n \sim \mathcal{U}\{2, \dots, 5\}$ annuli centred on the zero frequency, of inner radius $r \sim \mathcal{U}(0.1, 0.9) \rho_{\max}$ and width $\mathcal{U}(0.05, 0.15) \rho_{\max}$, with $\rho_{\max} = \lfloor \min(H, W)/2 \rfloor$, are set to zero;
- sectors: with $n \sim \mathcal{U}\{2, \dots, 6\}$, the angular sectors $[2k\pi/n, 2k\pi/n + \pi/n)$, $k = 0, \dots, n-1$, of the polar angle $\text{atan2}(u - H/2, v - W/2) \in (-\pi, \pi]$ are set to zero; only the sectors that start below π , in one half of the centred spectrum, are non-empty.

The input being complex, its spectrum has no Hermitian symmetry, and masking a region of the spectrum has no symmetric counterpart to preserve. Removing lines, rings or sectors of frequencies suppresses structures of given orientations or scales while keeping the coarse content of the patch.

A.6 Rotation and flip

Definition A.6 (Rotation and flip). With probability $\frac{1}{2}$, the patch is rotated by $k \cdot 90^\circ$ with $k \sim \mathcal{U}\{1, 2, 3\}$; then, with probability $\frac{1}{2}$, it is flipped horizontally, vertically, or both, each with probability $\frac{1}{3}$.

In the pipeline, this transformation is applied with probability $\frac{1}{2}$ to the patch before the two views are built, so that both views share the same orientation.

Bibliography

- [1] Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1120–1128. PMLR, 2016. URL <https://proceedings.mlr.press/v48/arjovsky16.html>.
- [2] Jose Agustin Barrachina, Chengfang Ren, Gilles Vieillard, Christèle Morisseau, and Jean-Philippe Ovarlez. Real- and complex-valued neural networks for SAR image segmentation through different polarimetric representations. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 1456–1460. IEEE, 2022. doi: 10.1109/ICIP46576.2022.9897186.
- [3] Jose Agustin Barrachina, Chengfang Ren, Gilles Vieillard, Christèle Morisseau, and Jean-Philippe Ovarlez. Theory and implementation of complex-valued neural networks. arXiv:2302.08286, 2023. URL <https://arxiv.org/abs/2302.08286>.
- [4] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1597–1607. PMLR, 2020. URL <https://proceedings.mlr.press/v119/chen20j.html>.
- [5] Shane R. Cloude and Eric Pottier. A review of target decomposition theorems in radar polarimetry. *IEEE Transactions on Geoscience and Remote Sensing*, 34(2):498–518, 1996. doi: 10.1109/36.485127.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *CoRR*, abs/2010.11929, 2020. URL <https://arxiv.org/abs/2010.11929>.
- [7] Florian Eilers and Xiaoyi Jiang. Building blocks for a complex-valued transformer architecture. In *ICASSP 2023 – 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023. doi: 10.1109/ICASSP49357.2023.10095349.
- [8] Jérémy Fix, Quentin Gabot, X. Huy Nguyen, Joana Frontera-Pons, Chengfang Ren, and Jean-Philippe Ovarlez. torchcvnn: A PyTorch-based library to easily experiment with state-of-the-art complex-valued neural networks. In *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9. IEEE, 2025. doi: 10.1109/IJCNN64981.2025.11229081.
- [9] J. W. Goodman. Some fundamental properties of speckle. *Journal of the Optical Society of America*, 66(11):1145–1150, 1976. doi: 10.1364/JOSA.66.001145.
- [10] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *2020 IEEE/CVF Conference*

- on *Computer Vision and Pattern Recognition (CVPR)*, pages 9726–9735. IEEE, 2020. doi: 10.1109/CVPR42600.2020.00975.
- [11] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv:1606.08415, 2016. URL <https://arxiv.org/abs/1606.08415>.
 - [12] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, pages 646–661. Springer International Publishing, 2016. doi: 10.1007/978-3-319-46493-0_39.
 - [13] IETR. San Francisco polarimetric SAR datasets. <https://ietr-lab.univ-rennes1.fr/polsarpro-bio/san-francisco/>, 2026. Accessed 29 September 2026.
 - [14] Japan Aerospace Exploration Agency. PALSAR-2 level 1.1/2.1/1.5/3.1 CEOS SAR product format description. https://www.eorc.jaxa.jp/ALOS/en/alos-2/pdf/product_format_description/PALSAR-2_xx_Format_CEOS_E_g.pdf, 2021. Version of 6 December 2021.
 - [15] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7482–7491. IEEE, 2018. doi: 10.1109/CVPR.2018.00781.
 - [16] Ken Kreutz-Delgado. The complex gradient operator and the CR-calculus. arXiv:0906.4835, 2009. URL <https://arxiv.org/abs/0906.4835>.
 - [17] Jong-Sen Lee and Eric Pottier. *Polarimetric Radar Imaging: From Basics to Applications*. CRC Press, 2009. doi: 10.1201/9781420054989.
 - [18] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944. IEEE, 2017. doi: 10.1109/CVPR.2017.106.
 - [19] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2999–3007. IEEE, 2017. doi: 10.1109/ICCV.2017.324.
 - [20] Xu Liu, Licheng Jiao, and Fang Liu. PolSF: PolSAR image dataset on San Francisco. arXiv:1912.07259, 2019. URL <https://arxiv.org/abs/1912.07259>.
 - [21] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440. IEEE, 2015. doi: 10.1109/CVPR.2015.7298965.
 - [22] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://openreview.net/forum?id=Skq89Scxx>.
 - [23] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.

- [24] Alberto Moreira, Pau Prats-Iraola, Marwan Younis, Gerhard Krieger, Irena Hajnsek, and Konstantinos P. Papathanassiou. A tutorial on synthetic aperture radar. *IEEE Geoscience and Remote Sensing Magazine*, 1(1):6–43, 2013. doi: 10.1109/MGRS.2013.2248301.
- [25] NASA Earthdata. Synthetic aperture radar (SAR). <https://www.earthdata.nasa.gov/learn/earth-observation-data-basics/sar>, 2026. Accessed 29 September 2026.
- [26] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.
- [27] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, Lecture Notes in Computer Science, pages 234–241. Springer International Publishing, 2015. doi: 10.1007/978-3-319-24574-4_28.
- [28] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge, 2015. URL <https://arxiv.org/abs/1409.0575>.
- [29] Chiheb Trabelsi, Olexa Bilaniuk, Ying Zhang, Dmitriy Serdyuk, Sandeep Subramanian, João Felipe Santos, Soroush Mehri, Negar Rostamzadeh, Yoshua Bengio, and Christopher J. Pal. Deep complex networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://openreview.net/forum?id=H1T2hmZAb>.
- [30] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018. URL <https://arxiv.org/abs/1807.03748>.
- [31] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 12077–12090. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/64f1f27bf1b4ec22924fd0acb550c235-Abstract.html.